A Major Project On

AI- DRIVEN MACHINE LEARNING APPROCH FOR CLASSIFYING THE FLOODING AND NON FLOODING ZONES

For the Award of the Degree of

MASTER OF COMPUTER APPLICATIONS

SUBMITTED

BY

KEELUKUPPA INDHU

(HT NO: 240200001048)

Under the esteemed guidance of

Dr.M . Ussennaiah MCA , Ph.D

Associate Professor

DEPARTMENT OF COMPUTER SCIENCE

VIKRAMA SIMHAPURI UNIVERSITY

NELLORE-524320

FEBRUARY-2026

PROJECT TITLE

AI -DRIVEN MACHINE LEARNING APPROACH FOR CLASSIFYING THE FLOODING AND FLOODING ZONES

MASTER OF COMPUTER APPLICATIONS

SUBMITTED BY

KEELUKUPPA INDHU

Reg. No: 240200001048

Under the esteemed guidance of

Dr.M . Ussennaiah MCA , Ph.D

Associate Professor

ABSTRACT

Every year, millions of individuals are affected by flooding in Bangladesh resulting in loss of habitat, properties, and lives. Because the country experiences monsoon rains, flat lands, cyclones as well as glacier melt, it is highly prone to flooding. This work aimed to lessen the impact of flooding disasters and develop effective strategies for early warning systems. The intertwined weather patterns known as the atmospheric river (AR) of Bangladesh make use of topological data analysis (TDA) in connection with wavelet decomposition and unsupervised machine learning (k-means clustering) methods to pave the way for enhanced flood detection. We have pinpointed the most desired cluster group, which stands at k = 4 in our argument. In terms of the silhouette coefficient, this group had the highest values in Rangpur at 0.9016, Sylhet at 0.9028, Barisal at 0.9094, and Khulna at 0.9014. Our work was highly efficient and ranked at approximately 96% using SVMs. Later, we identified flooded and non-flooded regions with the same technique. The potential flood-prone periods were identified for all four regions based on optimal clustering results, as indicated by silhouette coefficient values. At any time when there is a possibility of flood occurrence, it can alert residents in the affected regions to move out to safer grounds; this could also stimulate appropriate actions from the risk managers.

1. INTRODUCTION

Bangladesh is among the most flood-prone regions across the entire continent of Asia consequently visible the negative effects of this natural catastrophe annually due to that. Every year the nation goes through harsh monsoons with the ultimate outcome being wide-scale flooding which affects huge numbers of inhabitants. As per Mallick et al., floods in Bangladesh are a cyclical and catastrophic natural occurrence that causes many fatalities, infrastructure damage, and settlement displacement. These floods have far-reaching implications on the nation's development progress in terms of socio-economics and may set it back by many years. Tackling this disaster involves the use of topological data analysis (TDA) to manage floods well and minimize their subsequent impacts [4]. Topological data analysis is a popular tool for data modeling and structuring, offering a unique approach to understanding complex data sets. It pro vides another way to find out how complicated datasets are structured.

Using TDA with flood data helps in revealing concealed patterns and connections that are usually ignored by usual solutions. Bangladesh can handle flood disasters by understanding the complicated spatial and temporal patterns of these calamities using topological analysis to create viable mitigation strategies. In statistics, topological data analysis (TDA) is a new field that employs algebraic topology methods for encoding the form and configuration of data. It is especially advantageous in addressing high-dimensional data which is difficult for conventional methods of analysis.

TDA is made of mathematical tools that collect intrinsic shape information from data sets, unlocking information otherwise not easily accessible. It means that we can build better models by knowing what this shape is. The understanding evolved from different fundamental works in the 1980s and 1990s and Topological Data Analysis began to take shape which now has current applications.

There are but a few works of literature that have applied the combined approach in TDA in flood control, yet those that do, such as and indicate its potential. Nevertheless, many researchers have explored flood control without applying TDA to study pattern recognition using complex data. These listed authors did not apply TDA in flood modeling and control focusing instead on more conventional methods. Where

clustering is part of data mining, picking the right cluster count is key to improving predictive model accuracy for large datasets.

Unsupervised machine learning (ML) algorithm K-means is popular for its simplicity and effective ness. The method usually finds discrete, non-overlapping clusters with groups for each location, helping to identify regions with similar flood characteristics. The validation of the results obtained by clustering algorithms is a funda mental part of the clustering process, ensuring the reliability of the findings. The most used approaches for cluster validation are based on internal cluster validity indices. Although many indices have been proposed, there is no recent extensive comparative study of their performance.

For cluster validity tests, when we cluster the dataset for the parameter, we got the best cluster group at k = 4 This indicates an optimal number of clusters for the given data, allowing for a more nuanced understanding of flood patterns. The current study critically examined the cluster group using unsupervised machine learning, silhouette analysis, and silhouette score.

These methods helped determine the quality of the clustering by measuring how similar each point is to its own cluster compared to other clusters. In contrast to earlier research that exclusively employed supervised machine learning due to the natural occurrence, it is tough to identify the flooding zones, and the model performance accuracy was also too low. The limitations of supervised learning in this context stem from the difficulty in obtaining labeled data and the dynamic nature of flood events. This study is crucial to improve our analysis' accuracy and overcome the limitations of earlier research. Using Wavelet and unsupervised machine learning in flood study is a new approach, and by using this, our model performs very well and can find out the flooding and non-flooding zones in Bangladesh.

2. LITERATURE SURVEY

Flooding remains a significant natural hazard worldwide, posing threats to human lives, infrastructure, and ecosystems. Conventional flood detection methods often rely on remote sensing techniques, hydrological models, and statis tical approaches, which may have limitations in accurately predicting and monitoring flood events. To enhance flood detection capabilities, recent research has explored innovative approaches leveraging advances in data science and machine learning. Combining unsupervised machine learning methods with topological data analysis (TDA)is one such method that is gaining popularity for identifying atmospheric river (AR) patterns, which are strongly linked to floods.

The global hydrological cycle depends heavily on atmospheric rivers (ARs), which are thin regions of accelerated water vapor transport in the atmosphere that frequently span great distances. When these events combine with land masses, they can produce severe precipitation, which can cause floods invulnerable areas. Several research has shown the connection between AR activity and flood events, highlighting the necessity of reliable techniques to recognize and track AR patterns to manage floods effectively.

TDA is a mathematical framework for analyzing the shape and structure of data, particularly in high dimensional spaces. This approach has gained attention in various disciplines for its ability to capture complex topo logical features that traditional methods may overlook. In the context of atmospheric science, TDA offers a promising avenue for characterizing the intricate spatial and temporal dynamics of AR patterns. Unsupervised machine learning techniques play a pivotal role in extracting patterns and insights from data without explicit supervision. K-means and DBSCAN are two popular clustering techniques that are used to group similar data items according to their inherent properties.

Principal component analysis (PCA) and t-distributed stochastic neighbor embedding (t-SNE) are two dimensionality reduction approaches that make it possible to see and explore high-dimensional data in lower-dimensional spaces. These methods offer valuable tools for identifying and categorizing AR patterns from atmospheric data with minimal human intervention.

The integration of TDA with unsupervised machine learning techniques presents a synergistic approach to AR recognition and flood detection. By representing atmospheric data in a topologically meaningful manner, TDA facilitates the

identification of critical features associated with AR activity. Sub sequent application of unsupervised clustering algorithms enables the classification of AR patterns into distinct categories based on their topological signatures. This integrated framework enhances the understanding of AR dynamics and provides valuable insights for flood prediction and risk assessment. Several studies have demonstrated the efficacy of the proposed approach in real-world set tings.

For instance, applied TDA coupled with K-means clustering to atmospheric datasets, successfully identifying different types of AR patterns and their correlations with historical flood events. Similarly, employed PCA and hierarchical clustering to analyze AR behavior, leading to improved flood forecasting capabilities in coastal regions. These case studies underscore the potential of TDA-based approaches in enhancing flood detection and early warning systems. Despite the promising results, several challenges remain in the application of TDA and unsupervised learning for AR recognition.

Data quality, computational complexity, and model interpretability are among the key considerations that warrant further investigation. Future research efforts should focus on refining the proposed methodologies, integrating multi-source data sources, and developing robust validation frameworks to ensure the reliability and scalability of flood detection systems. Jiang et al. highlight the importance of integrating optical, SAR, and LiDAR data for improved change detection and flood prediction. Their study emphasizes multisource fusion to enhance environmental monitoring.

Similarly, Dai et al. propose a CNN-based water segmentation method for SAR images, improving accuracy in flood assessment and coastal monitoring. To sum up, the combination of unsupervised machine learning methods and topological data analysis offers a fresh and exciting method for identifying atmospheric river patterns in flood detection applications. This technique improves flood prediction and risk assessment capabilities by utilizing the intrinsic structure of atmospheric data to identify important elements linked to AR activity.

3. METHODOLOGY

3.1 Data Collection

Data is gathered from multiple reliable sources such as:

- Remote sensing data including rainfall, elevation, and soil moisture
- Digital Elevation Models (DEM) for terrain analysis
- Historical flood records and hydrological datasets
- Weather and climate data from meteorological departments

3.2 Data Preprocessing

Before analysis, the collected data undergoes preprocessing:

- **Data Cleaning:** Removing noise, missing values, and corrupted satellite images
- **Data Alignment:** Georeferencing and aligning spatial datasets from different sources
- **Normalization:** Scaling features such as rainfall, elevation, and water levels
- **Feature Extraction:** Extracting relevant features like slope, drainage density, and vegetation index (NDVI)

3.3 Data Storage

Processed data is stored in a structured system:

- Geospatial databases for spatial data handling
- Raster and vector data formats for satellite and map-based data
- Data is organized into layers such as:
- Terrain/Elevation Data
- Hydrological Data
- Climate Data
- Land Use/Land Cover Maps

3.4 Data Processing and Management

The system processes stored data using advanced analytical techniques:

- **Wavelet Transform:** Applied to decompose spatial and temporal data into multi-resolution components for detecting flood patterns
- **Topological Data Analysis (TDA):** Used to capture shape-based features and spatial relationships (e.g., connected regions, holes, clusters)
- **Unsupervised Learning:** Clustering algorithms (e.g., K-means, DBSCAN) to group regions into flooding and non-flooding zones
- **Feature Integration:** Combining wavelet features and topological features for improved pattern recognition

3.5 Data Security and Privacy

To ensure reliability and protection of data:

- Secure storage of geospatial datasets using access control mechanisms
- Data validation to ensure accuracy of spatial and environmental data
- Backup systems to prevent loss of critical flood-related data
- Version control for tracking updates in datasets

3.6 Data Retrieval and Presentation

Results are retrieved and presented through:

- GIS-based visualization maps highlighting flood-prone and safe zones
- Interactive dashboards for analyzing spatial patterns
- Web applications for public and administrative use

3.7 Data Updating and Maintenance

- Regular updates with new satellite imagery and weather data
- Continuous model refinement with recent flood events
- Integration with real-time data sources (e.g., rainfall sensors, IoT devices)
- Periodic validation of model predictions with ground truth data
- Maintenance of spatial databases to ensure consistency and accuracy

3.8 PROPOSED SYSTEM

- The proposed system employs a hybrid approach combining topological data analysis (TDA), wavelet decomposition, and unsupervised machine learning (k-means clustering) to enhance flood detection in Bangladesh.
- By analyzing atmospheric river patterns and identifying key cluster groups, the system accurately classifies flooded and non-flooded regions and achieves high efficiency using SVMs.
- It is designed to provide early warnings to residents and support risk managers in taking timely preventive actions, thereby reducing the impact of flooding disasters.

4. SYSTEM REQUIREMENTS AND SPECIFICATIONS

4.1 Hardware Requirement

- Processor : I3
- RAM : 8 GB
- Hard Disk : 500 GB

4.2 Software Requirement

- Operating System : Window 10
- Programming Language : Python
- Front End : HTML, CSS
- Back End : MySQL

5. REQUIREMENT ANALYSIS

5.1 ARCHITECTURE

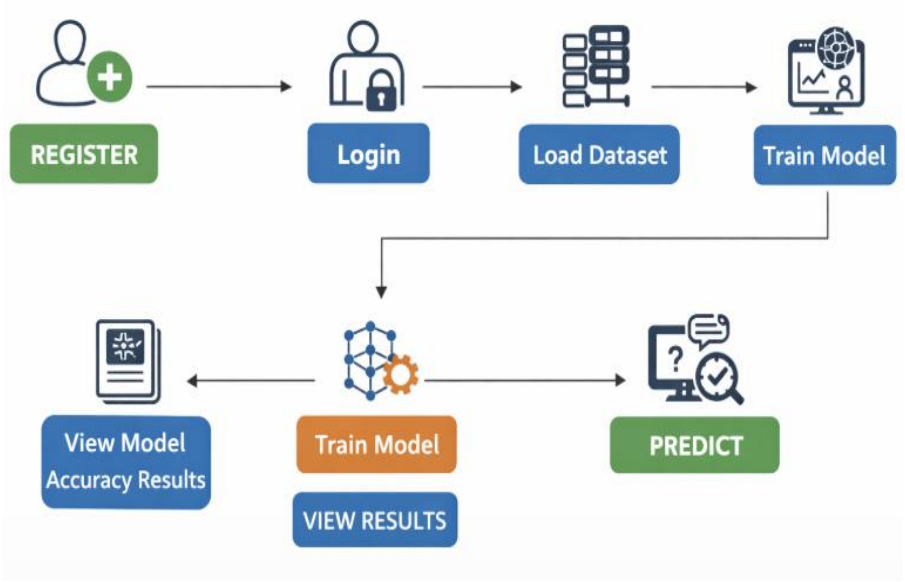


Fig. 5.1: System Architecture

5.2 UML DIAGRAMS

The overall logical structure of a database can be expressed graphically by an **E-R diagram.** The relative simplicity and pictorial clarity of this diagramming technique may well account in large part for the widespread use of the E-R model. Such a diagram consists of the following major components.

Rectangles: Represent Entity Sets.

Ellipses: Represent attributes.

Diamonds: Represent relationship sets

Lines: Link attributes to entity sets and entity sets

DATA FLOW DIAGRAM

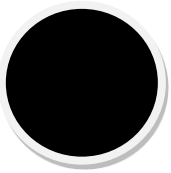
A graphical tool used to describe and analyze the moment of data through a system manual or automated including the process, stores of data, and delays in the system. Data Flow Diagrams are the central tool and the basis from which other components are developed. The transformation of data from input to output, through processes, may

be described logically and independently of the physical components associated with the system. The DFD is also know asa data flow graph or a bubble chart.

DFDs are the model of the proposed system. They clearly should show the requirements on which the new system should be built. Later during design activity this is taken as the basis for drawing the system's structure charts. The Basic Notation used to create a DFD's are as follows:

1. **Dataflow:** Data move in a specific direction from an origin to a destination.

Process: People, procedures, or devices that use or produce (Transform) Data. The physical component is not identified.



3. Source: External sources or destination of data, which may be People, programs, organizations or other entities.

4. **Data Store:** Here data are stored or referenced by a process in the System.

UML DIAGRAMS:

Unified Modeling Language is a language available to perform modeling of software. A model is simplification of reality. A model provides the blue print of the system, model encompasses detailed plans.

Building blocks of the UML

The vocabulary of the UML encompasses three kinds of building blocks.

1. Things.
2. Relationships.
3. Diagrams.

Things in the UML

Things are the abstractions that are first-class citizen in a model.

There are four kinds of things in the UML.

1. Structure things.
2. Behavioral things.
3. Grouping things.
4. Annotational things.

These things are the basic object-oriented building blocks of the UML. You use them to write well-formed models.

Relationships in the UML

Things can be connected to logically are physically with the help of relationship in object oriented modeling. These are four kinds of relationships in the UML.

1. Dependency.
2. Association.
3. Generalization.
4. Realization.

USECASE DIAGRAM

Use case diagram shows a set of usecases and actors (a special kind of class) and their relationship. Usecase diagrams address the static usecase view of a system.

These diagrams are especially important in organizing and modeling the behavioral of a system both sequence and collaboration diagrams are kind of interaction diagram.

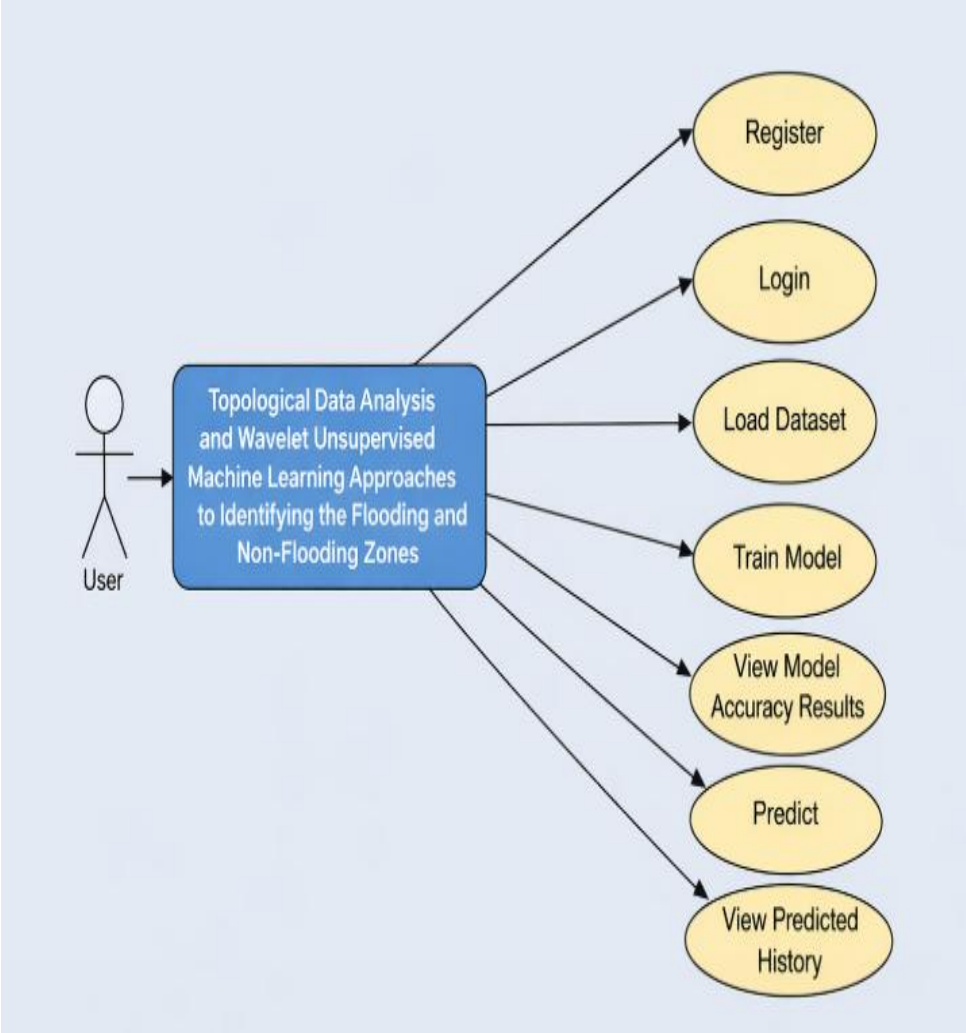


Fig 5.2 Use-case Diagram

CLASS DIAGRAM:

Class diagrams area unit the foremost common diagrams employed in UML. Category diagram consists of categories, interfaces, associations and collaboration. Category diagrams primarily represent the thing directed read of a system that is static in nature. Active category is employed in a very category diagram to represent the concurrency of the system.

Class diagram represents the thing orientation of a system. Therefore it's usually used for development purpose. This can be the foremost wide used diagram at the time of system construction.

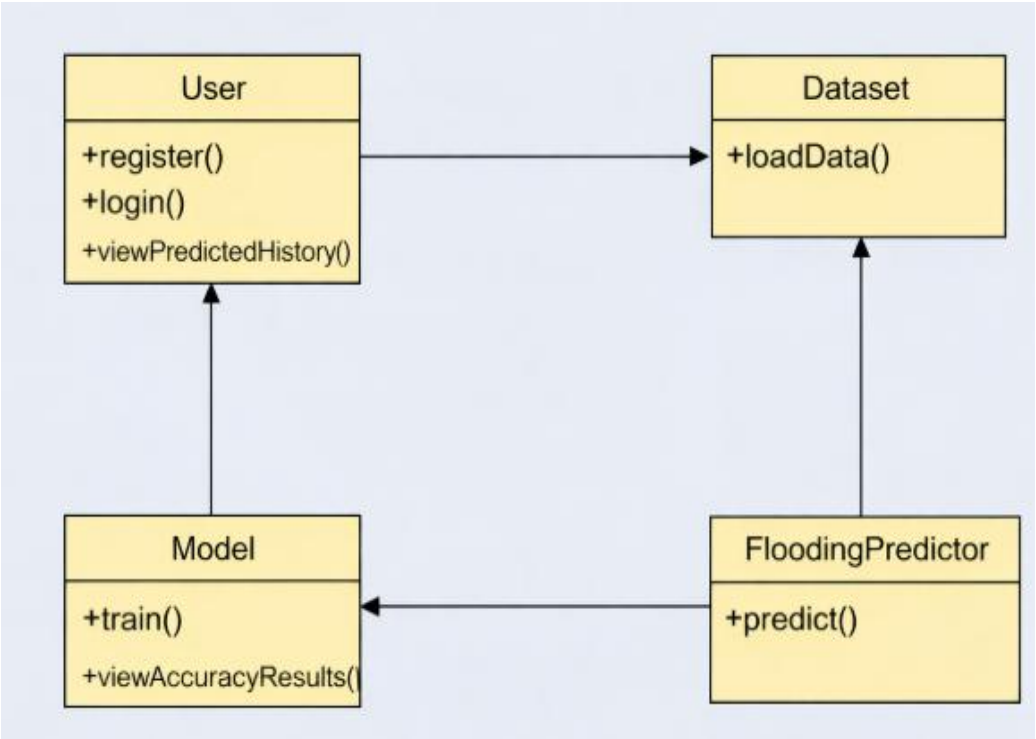


Fig 5.2 Class Diagram

SEQUENCE DIAGRAM

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

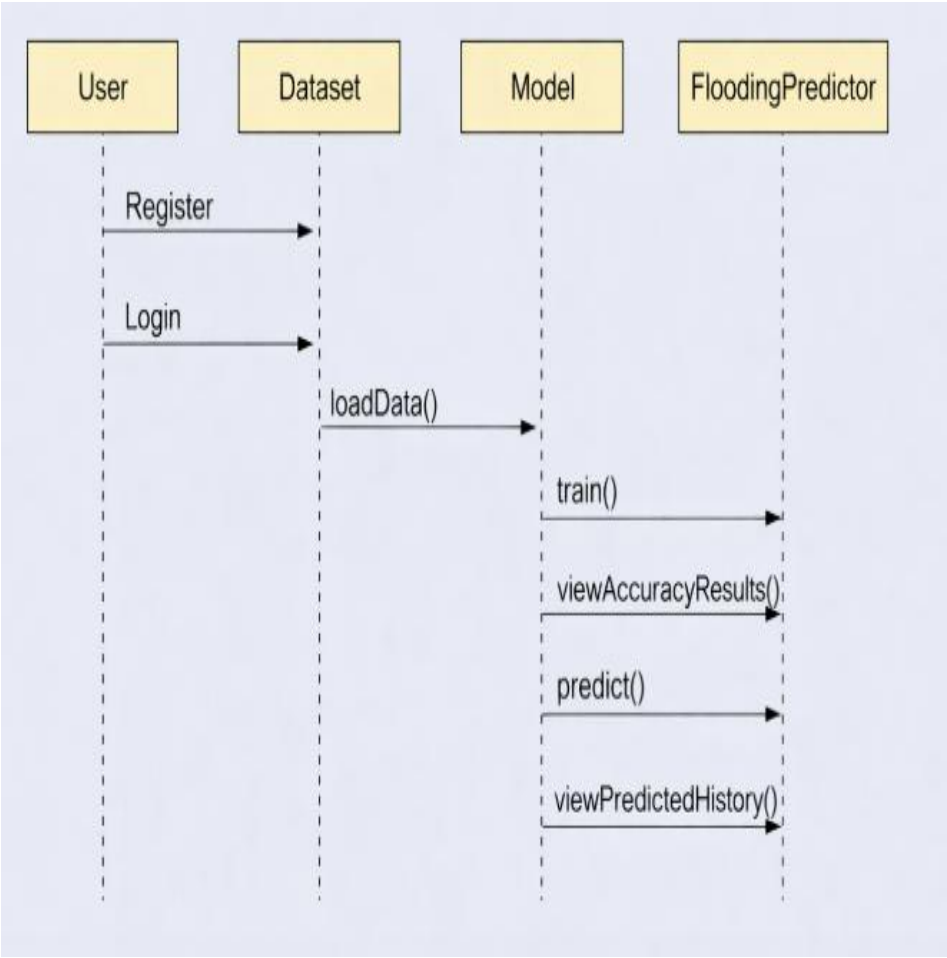


Fig 5.3 Sequence Diagram

ACTIVITY DIAGRAM

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control

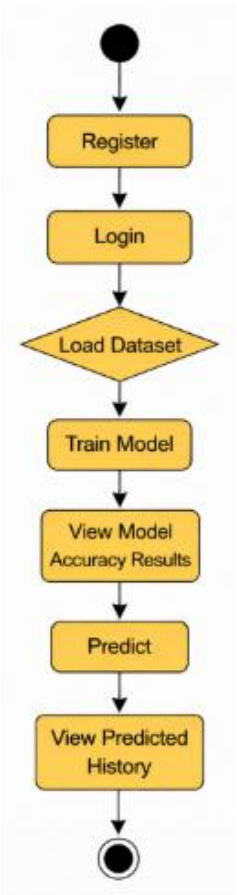


Fig. 5.4: Activity Diagram

MODULE DESCRIPTION:

User Module

- Handles user interaction with the system.
- Allows users to register, log in, and access all functionalities like prediction and history viewing.

Dataset Module

- Responsible for uploading and managing datasets.
- Supports loading of geographical or environmental data required for flood prediction.

Model Training Module

- Implements unsupervised machine learning algorithms.
- Trains the model using processed data to identify flooding and non-flooding zones.

Prediction Module

- Generates predictions for new input data.
- Classifies regions as flooding or non-flooding based on trained model.

Algorithms

Logistic Regression

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, it gives the probabilistic values which lie between 0 and 1.

Decision tree

- Trees are constructed through an algorithmic approach that identifies ways to split the data set based on different conditions.
- It is one of the most widely used practical methods for supervised learning.
- These are non-parametric method used for both classification and regression.

Random forest

- It generates multi decision trees from which each decision tree uses a part of data sample and predicts the result.
- Random forest is a Supervised Learning algorithm which uses ensemble learning method for classification and regression.
- Random forest is a bagging technique and the trees in random forests run in parallel without any interactions.

6. TESTING & VALIDATION

Software Testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding, Testing presents an interesting anomaly for the software engineer

Testing Objectives include:

1. Testing is a process of executing a program with the intent of finding an error
2. A good test case is one that has a probability of finding an as yet undiscovered error
3. A successful test is one that uncovers an undiscovered error

Testing Principles:

- All tests should be traceable to end user requirements
- Tests should be planned long before testing begins
- Testing should begin on a small scale and progress towards testing in large
- Exhaustive testing is not possible
- To be most effective testing should be conducted by a independent third party

Testing Strategies

A Strategy for software testing integrates software test cases into a series of well planned steps that result in the successful construction of software. Software testing is a broader topic for what is referred to as Verification and Validation. Verification refers to the set of activities that ensure that the software correctly implements a specific function Validation refers he set of activities that ensure that the software that has been built is traceable to customer's requirements.

Unit Testing:

Unit testing focuses verification effort on the smallest unit of software design that is the module. Using procedural design description as a guide, important control paths are tested to uncover errors within the boundaries of the module. The unit test is normally white box testing oriented and the step can be conducted in parallel for multiple modules.

Integration Testing:

Integration testing is a systematic technique for constructing the program structure while conducting test to uncover errors associated with the interfacing. The objective is to take unit tested methods and build a program structure that has been dictated by design.

Top-down Integration:

Top down integrations an incremental approach to construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main control program. Modules subordinate to the main program are incorporated in the structure either in the breath-first or depth-first manner.

Bottom-up Integration:

This method as the name suggests, begins construction and testing with atomic modules i.e., modules at the lowest leveling the program structure. Because the modules are integrated in the bottom up manner the processing required for the modules subordinate to a given level is always available and the need for stubs is eliminated.

Validation Testing:

At the end of integration testing software ids completely assembled as a package. Validation testing is the next stage which can be defined as successful when the software functions in the manner reasonably expected by the customer. Reasonable expectations are those defined in the software requirements specifications. Information contained in those sections form a basis for validation testing approach.

System Testing:

System testing is actually a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated to perform allocated function.

7. IMPLEMENTATION& RESULTS

7.1 INTRODUCTON

This mapping permits forward engineering: The generation of code from a UML model into a programming language. The reverse is also possible: You can reconstruct a model from an implementation back into the UML. Reverse engineering is not magic. Unless you encode that information in the implementation, information is lost when moving forward from models to code. Reverse engineering thus requires tool support with human intervention.

7.2 TECHNOLOGY DESCRIPTION

Introduction to Python

Python is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including structured (particularly, procedural), object-oriented, and functional programming. Python is often described as a "batteries included" language due to its comprehensive standard library. Python was conceived in the late 1980s as a successor to the ABC language. Python 2.0, released in 2000, introduced features like list comprehensions and a garbage collection system capable of collecting reference cycles. Python 3.0, released in 2008, was a major revision of the language that is not completely backward-compatible, and much Python 2 code does not run unmodified on Python 3. The Python 2 language, i.e. Python 2.7.x, was officially discontinued on 1 January 2020 (first planned for 2015) after which security patches and other improvements will not be released for it.[32][33] With Python 2's end-of-life, only Python 3.5.x and later are supported. Python interpreters are available for many operating systems. A global community of programmers develops and maintains CPython, an open source[35] reference implementation. A non-profit organization, the Python Software Foundation, manages and directs resources for Python and CPython development.

SYNTAX AND SEMANTICS

Python is meant to be an easily readable language. Its formatting is visually uncluttered, and it often uses English keywords where other languages use punctuation.

Unlike many other languages, it does not use curly brackets to delimit blocks, and semicolons after statements are optional. It has fewer syntactic exceptions and special cases than C or Pascal.

Indentation

Main article: Python syntax and semantics § Indentation

Python uses whitespace indentation, rather than curly brackets or keywords, to delimit blocks. An increase in indentation comes after certain statements; a decrease in indentation signifies the end of the current block. Thus, the program's visual structure accurately represents the program's semantic structure. This feature is sometimes termed the off-side rule, which some other languages share, but in most languages indentation doesn't have any semantic meaning.

Statements and control flow

Python's statements include (among others):

The assignment statement (token '=', the equals sign). This operates differently than in traditional imperative programming languages, and this fundamental mechanism (including the nature of Python's version of variables) illuminates many other features of the language. Assignment in C, e.g., x = 2, translates to "typed variable name x receives a copy of numeric value 2". The (right-hand) value is copied into an allocated storage location for which the (left-hand) variable name is the symbolic address. The memory allocated to the variable is large enough (potentially quite large) for the declared type. In the simplest case of Python assignment, using the same example, x = 2, translates to "(generic) name x receives a reference to a separate, dynamically allocated object of numeric (int) type of value 2." This is termed binding the name to the object. Since the name's storage location doesn't contain the indicated value, it is improper to call it a variable.

Names may be subsequently rebound at any time to objects of greatly varying types, including strings, procedures, complex objects with data and methods, etc. Successive assignments of a common value to multiple names, e.g., x = 2; y = 2; z = 2 result in allocating storage to (at most) three names and one numeric object, to which all three names are bound.

Since a name is a generic reference holder it is unreasonable to associate a fixed data type with it. However at a given time a name will be bound to some object, which will have a type; thus there is dynamic typing.

- The if statement, which conditionally executes a block of code, along with else and elif (a contraction of else-if).
- The for statement, which iterates over an iterable object, capturing each element to a local variable for use by the attached block.
- The while statement, which executes a block of code as long as its condition is true.
- The try statement, which allows exceptions raised in its attached code block to be caught and handled by except clauses; it also ensures that clean-up code in a finally block will always be run regardless of how the block exits.
- The raise statement, used to raise a specified exception or re-raise a caught exception.
- The class statement, which executes a block of code and attaches its local namespace to a class, for use in object-oriented programming.
- The def statement, which defines a function or method.
- The with statement, from Python 2.5 released in September 2006, which encloses a code block within a context manager (for example, acquiring a lock before the block of code is run and releasing the lock afterwards, or opening a file and then closing it), allowing Resource Acquisition Is Initialization (RAII)-like behavior and replaces a common try/finally idiom.
- The break statement, exits from the loop.
- The continue statement, skips this iteration and continues with the next item.
- The pass statement, which serves as a NOP. It is syntactically needed to create an empty code block.
- The assert statement, used during debugging to check for conditions that ought to apply.
- The yield statement, which returns a value from a generator function. From Python 2.5, yield is also an operator. This form is used to implement coroutines.

The import statement, which is used to import modules whose functions or variables can be used in the current program. There are three ways of using import:

import <module name> [as <alias>] or from <module name> import * or from <module name> import <definition 1> [as <alias 1>], <definition 2> [as <alias 2>],

The print statement was changed to the print() function in Python 3.

Python does not support tail call optimization or first-class continuations, and, according to Guido van Rossum, it never will. However, better support for coroutine-like functionality is provided in 2.5, by extending Python's generators. Before 2.5, generators were lazy iterators; information was passed unidirectionally out of the generator. From Python 2.5, it is possible to pass information back into a generator function, and from Python 3.3, the information can be passed through multiple stack levels.

EXPRESSIONS

Some Python expressions are similar to languages such as C and Java, while some are not:

Addition, subtraction, and multiplication are the same, but the behavior of division differs. There are two types of divisions in Python. They are floor division (or integer division) // and floating point/division. Python also added the ** operator for exponentiation.

From Python 3.5, the new @ infix operator was introduced. It is intended to be used by libraries such as NumPy for matrix multiplication.

From Python 3.8, the syntax :=, called the 'walrus operator' was introduced. It assigns values to variables as part of a larger expression.

In Python, == compares by value, versus Java, which compares numerics by valueand objects by reference. (Value comparisons in Java on objects can be performed with the equals() method.) Python's is operator may be used to compare object identities (comparison by reference). In Python, comparisons may be chained, for example a <= b <= c.

Python uses the words and, or, not for its boolean operators rather than the symbolic &&, ||, !used in Java and C.

Python has a type of expression termed a list comprehension. Python 2.4 extended list comprehensions into a more general expression termed a generator expression.

Anonymous functions are implemented using lambda expressions; however, these are limited in that the body can only be one expression.

Conditional expressions in Python are written as x if c else y (different in order of operands from the c ? x : y operator common to many other languages).

Python makes a distinction between lists and tuples. Lists are written as [1, 2, 3], are mutable, and cannot be used as the keys of dictionaries (dictionary keys must be immutable in Python). Tuples are written as (1, 2, 3), are immutable and thus can be used as the keys of dictionaries, provided all elements of the tuple are immutable. The + operator can be used to concatenate two tuples, which does not directly modify their contents, but rather produces a new tuple containing the elements of both provided tuples. Thus, given the variable t initially equal to (1, 2, 3), executing t = t + (4, 5) first evaluates t + (4, 5), which yields (1, 2, 3, 4, 5), which is then assigned back to t, thereby effectively "modifying the contents" of t, while conforming to the immutable nature of tuple objects. Parentheses are optional for tuples in unambiguous contexts.

Python features sequence unpacking wherein multiple expressions, each evaluating to anything that can be assigned to (a variable, a writable property, etc.), are associated in the identical manner to that forming tuple literals and, as a whole, are put on the left hand side of the equal sign in an assignment statement.

The statement expects an iterable object on the right hand side of the equal sign that produces the same number of values as the provided writable expressions when iterated through, and will iterate through it, assigning each of the produced values to the corresponding expression on the left.

Python has a "string format" operator %. This functions analogous to printf format strings in C, e.g. "spam=%s eggs=%d" % ("blah", 2) evaluates to "spam=blah eggs=2".

In Python 3 and 2.6+, this was supplemented by the format() method of the str class, e.g. "spam={0} eggs={1}".format("blah", 2). Python 3.6 added "f-strings": blah = "blah"; eggs = 2; f'spam={blah} eggs={eggs}'.

Python has various kinds of string literals:

Strings delimited by single or double quote marks. Unlike in Unix shells, Perl and Perl-influenced languages, single quote marks and double quote marks function identically. Both kinds of string use the backslash (\) as an escape character. String interpolation became available in Python 3.6 as "formatted string literals".

Triple-quoted strings, which begin and end with a series of three single or double quote marks. They may span multiple lines and function like here documents in shells, Perl and Ruby.

Raw string varieties, denoted by prefixing the string literal with an r. Escape sequences are not interpreted; hence raw strings are useful where literal backslashes are common, such as regular expressions and Windows-style paths. Compare "@-quoting" in C#.

Python has array index and array slicing expressions on lists, denoted as a[key], a[start:stop] or a[start:stop:step]. Indexes are zero-based, and negative indexes are relative to the end. Slices take elements from the start index up to, but not including, the stop index. The third slice parameter, called step or stride, allows elements to be skipped and reversed. Slice indexes may be omitted, for example a[:] returns a copy of the entire list. Each element of a slice is a shallow copy.

In Python, a distinction between expressions and statements is rigidly enforced, in contrast to languages such as Common Lisp, Scheme, or Ruby. This leads to duplicating some functionality. For example:

List comprehensions vs. for-loops

Conditional expressions vs. if blocks

The eval() vs. exec() built-in functions (in Python 2, exec is a statement); the former is for expressions, the latter is for statements.

Statements cannot be a part of an expression, so list and other comprehensions or lambda expressions, all being expressions, cannot contain statements. A particular case of this is that an assignment statement such as a = 1 cannot form part of the conditional expression of a conditional statement. This has the advantage of avoiding a classic C error of mistaking an assignment operator = for an equality operator == in conditions: if (c = 1) { ... } is syntactically valid (but probably unintended) C code but if c = 1: ... causes a syntax error in Python.

METHODS

Methods on objects are functions attached to the object's class; the syntax instance. method(argument) is, for normal methods and functions, syntactic sugar for Class. method(instance, argument). Python methods have an explicit self parameter to access instance data, in contrast to the implicit self (or this) in some other object-oriented programming languages (e.g., C++, Java, Objective-C, or Ruby).

APPLICATIONS OF PYTHON

As mentioned before, Python is one of the most widely used language over the web. I'm going to list few of them here:

Easy-to-learn − Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.

Easy-to-read − Python code is more clearly defined and visible to the eyes.

Easy-to-maintain − Python's source code is fairly easy-to-maintain.

A broad standard library − Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.

Interactive Mode − Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.

Portable − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.

Extendable − You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.

Databases − Python provides interfaces to all major commercial databases.

GUI Programming − Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.

Scalable − Python provides a better structure and support for large programs than shell scripting.

INSTALLATION STEPS OF PYTHON

Installing and using Python on Windows 10 is very simple. The installation procedure involves just three steps:

- Download the binaries
- Run the Executable installer
- Add Python to PATH environmental variables

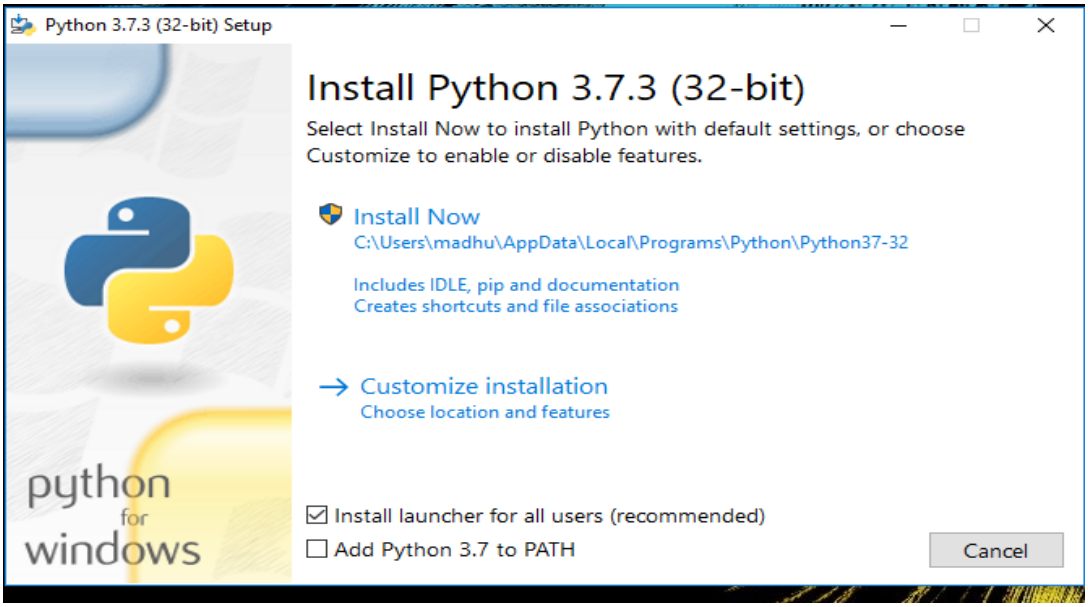
To install Python, you need to download the official Python executable installer. Next, you need to run this installer and complete the installation steps. Finally, you can configure the PATH variable to use python from the command line.

Step 1: Download the Python Installer binaries

- Open the official Python website in your web browser. Navigate to the Downloads tab for Windows.
- Choose the latest Python 3 release. In our example, we choose the latest Python 3.7.3 version. Click on the link to download Windows x86 executable installer if you are using a 32-bit installer.
- In case your Windows installation is a 64-bit system, then download Windows x86-64 executable installer.

Step 2: Run the Executable Installer

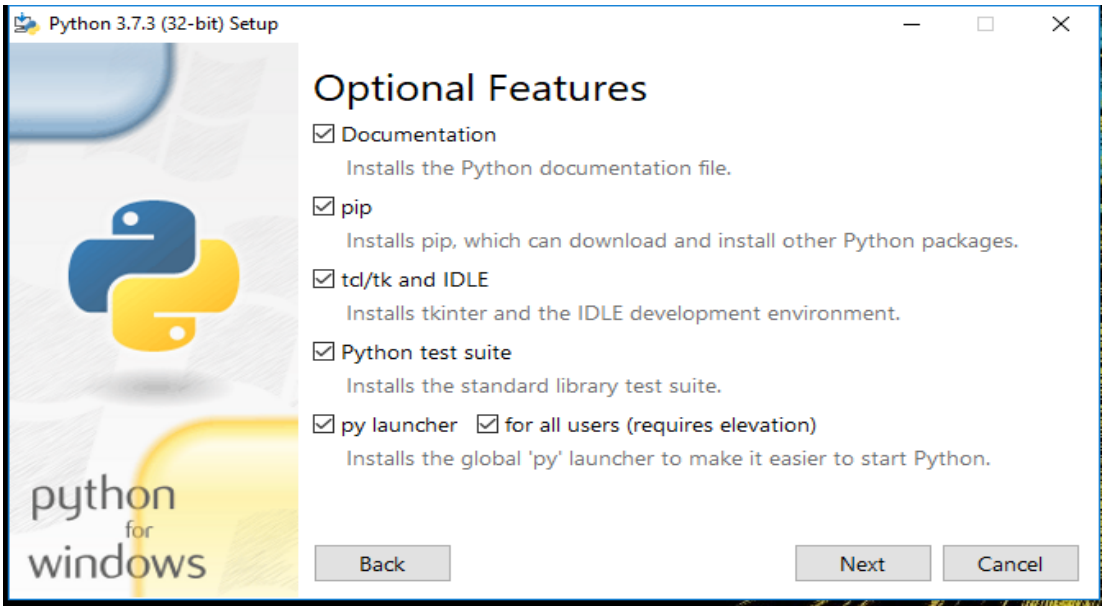
1. Once the installer is downloaded, run the Python installer.
2. Check the Install launcher for all users check box. Further, you may check the Add Python 3.7 to path check box to include the interpreter in the execution path.



3. Select **Customize installation**.

Choose the optional features by checking the following check boxes:

1. Documentation
2. pip
3. tcl/tk and IDLE (to install tkinter and IDLE)
4. Python test suite (to install the standard library test suite of Python)
5. Install the global launcher for `.py` files. This makes it easier to start Python
6. Install for all users.

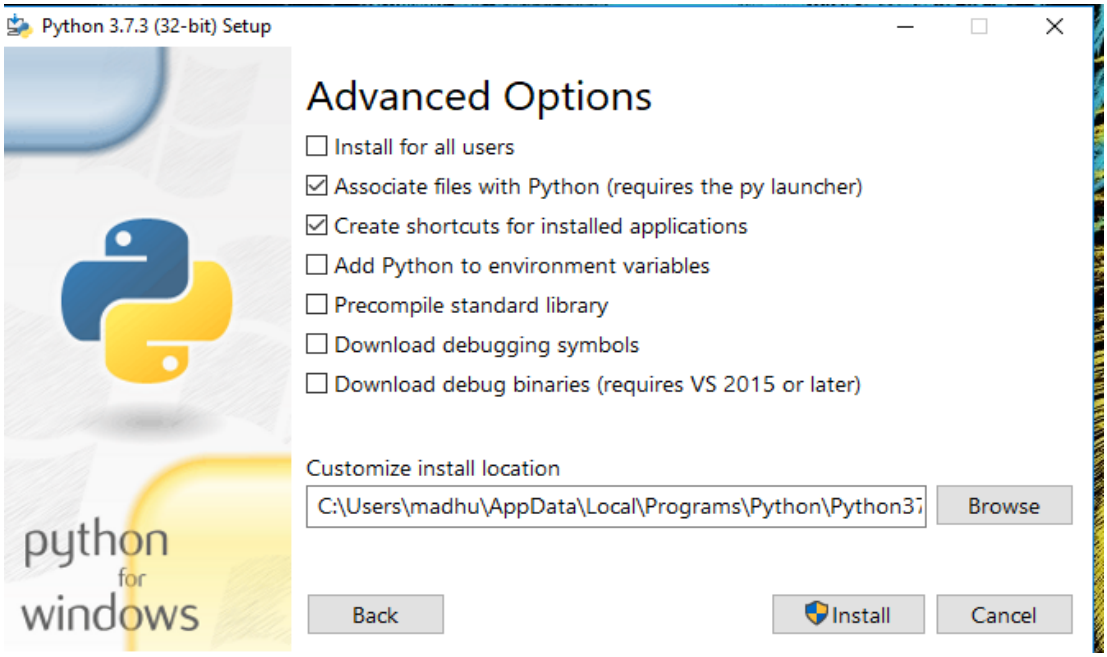


Click Next.

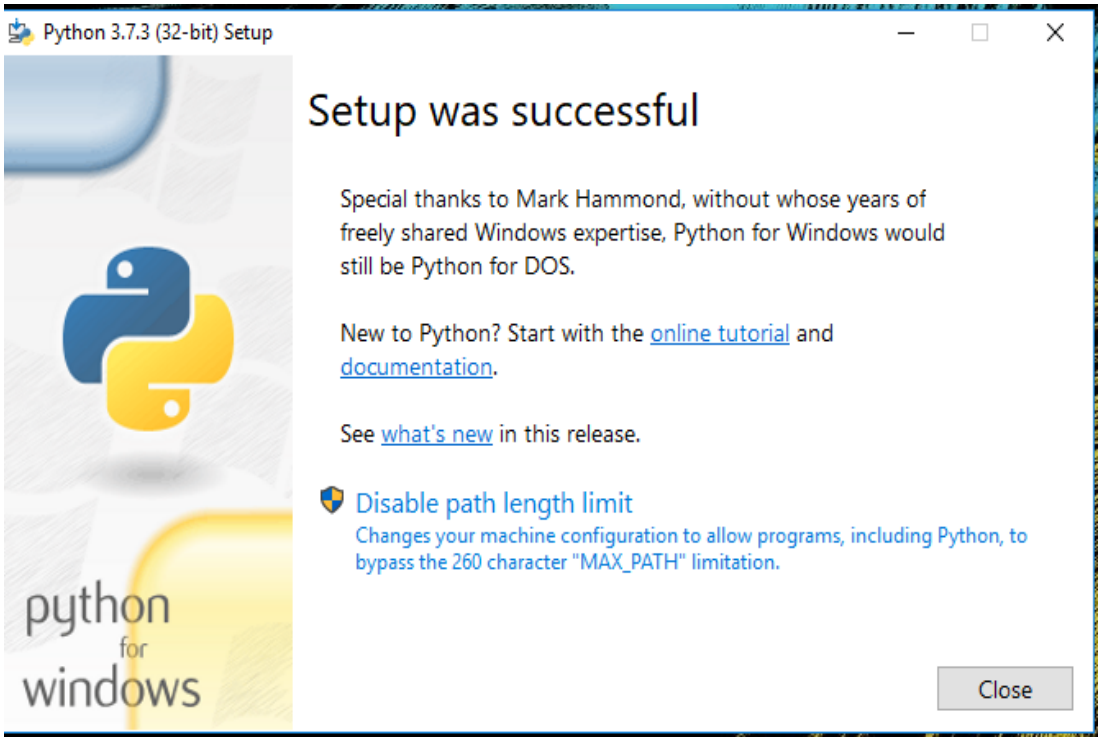
4. This takes you to Advanced Options available while installing Python. Here, select the Install for all users and Add Python to environment variables check boxes.

Optionally, you can select the Associate files with Python, Create shortcuts for installed applications and other advanced options. Make note of the python installation directory displayed in this step. You would need it for the next step.

After selecting the Advanced options, click Install to start installation.



5. Once the installation is over, you will see a Python Setup Successful window.



Step 3: Add Python to environmental variables

The last (optional) step in the installation process is to add Python Path to the System Environment variables. This step is done to access Python through the command line. In case you have added Python to environment variables while setting the Advanced options during the installation procedure, you can avoid this step. Else, this step is done manually as follows.

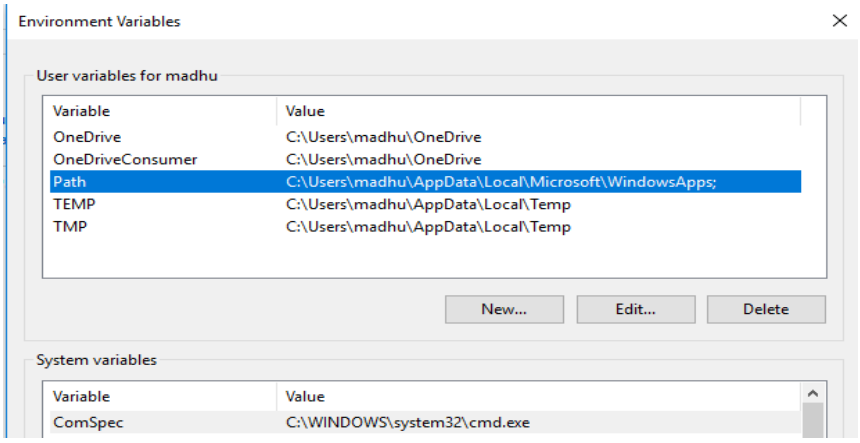
In the Start menu, search for "advanced system settings". Select "View advanced system settings". In the "System Properties" window, click on the "Advanced" tab and then click on the "Environment Variables" button.

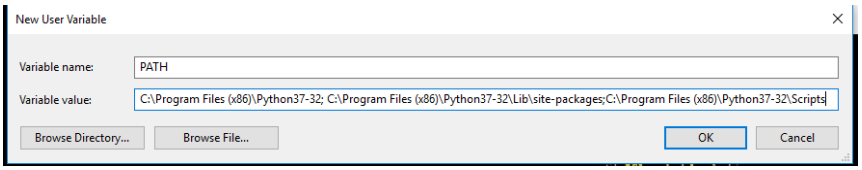
Locate the Python installation directory on your system. If you followed the steps exactly as above, python will be installed in below locations:

- C:\Program Files (x86)\Python37-32: for 32-bit installation
- C:\Program Files\Python37-32: for 64-bit installation

The folder name may be different from "Python37-32" if you installed a different version. Look for a folder whose name starts with Python.

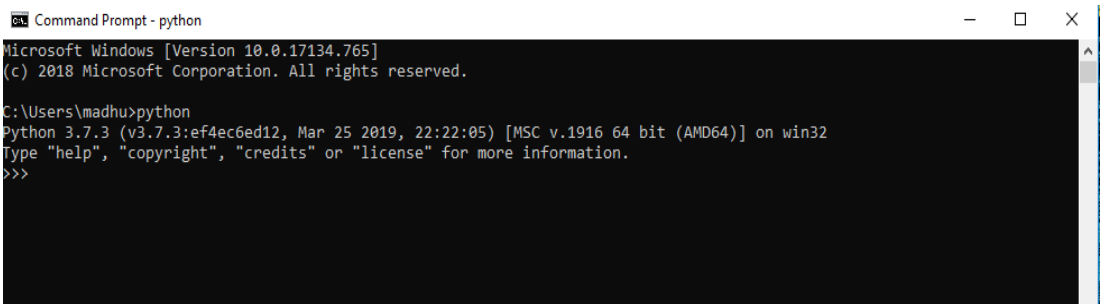
Append the following entries to PATH variable as shown below:



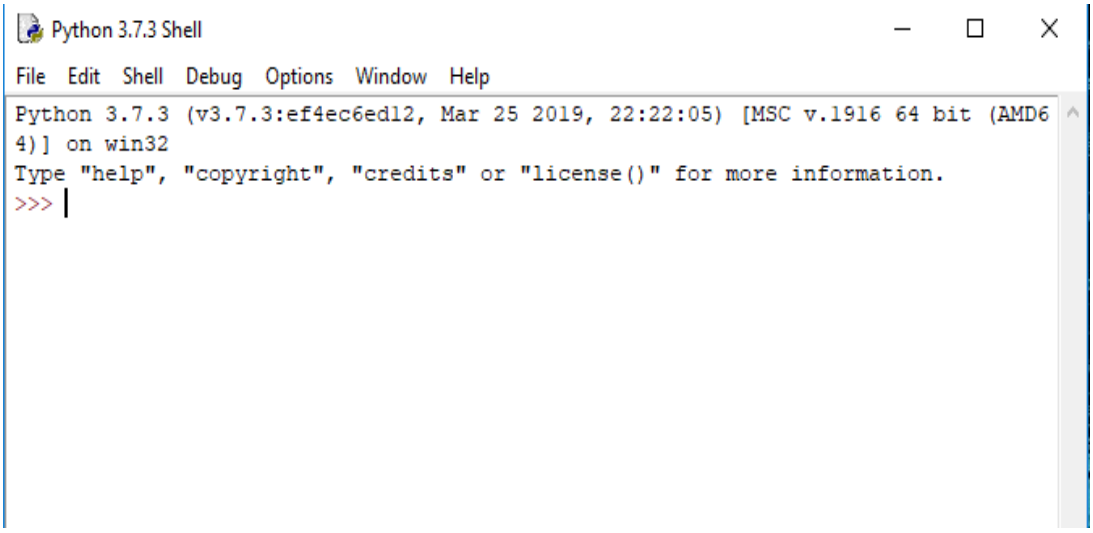


Step 4: Verify the Python Installation

You have now successfully installed Python 3.7.3 on Windows 10. You can verify if the Python installation is successful either through the command line or through the IDLE app that gets installed along with the installation. Search for the command prompt and type "python". You can see that Python 3.7.3 is successfully installed.



An alternate way to reach python is to search for "Python" in the start menu and clicking on IDLE (Python 3.7 64-bit). You can start coding in Python using the Integrated Development Environment(IDLE).



Uses

Since 2003, Python has consistently ranked in the top ten most popular programming languages in the TIOBE Programming Community Index where, as of February 2020, it is the third most popular language (behind Java, and C). It was selected Programming Language of the Year in 2007, 2010, and 2018.

- An empirical study found that scripting languages, such as Python, are more productive than conventional languages, such as C and Java, for programming problems involving string manipulation and search in a dictionary, and determined that memory consumption was often "better than Java and not much worse than C or C++".
- Large organizations that use Python include Wikipedia, Google, Yahoo!, CERN, NASA, Facebook, Amazon, Instagram, Spotify and some smaller entities like ILM and ITA. The social news networking site Reddit is written entirely in Python.
- Python can serve as a scripting language for web applications, e.g., via mod_wsgi for the Apache web server. With Web Server Gateway Interface, a standard API has evolved to facilitate these applications. Web frameworks like Django, Pylons, Pyramid, TurboGears, web2py, Tornado, Flask, Bottle and Zope support developers in the design and maintenance of complex applications. Pyjs and IronPython can be used to develop the client-side of Ajax-based applications.

- SQLAlchemy can be used as data mapper to a relational database. Twisted is a framework to program communications between computers, and is used (for example) by Dropbox.
- Libraries such as NumPy, SciPy and Matplotlib allow the effective use of Python in scientific computing, with specialized libraries such as Biopython and Astropy providing domain-specific functionality. SageMath is a mathematical software with a notebook interface programmable in Python: its library covers many aspects of mathematics, including algebra, combinatorics, numerical mathematics, number theory, and calculus.
- Python has been successfully embedded in many software products as a scripting language, including in finite element method software such as Abaqus, 3D parametric modeler like FreeCAD, 3D animation packages such as 3ds Max, Blender, Cinema 4D, Lightwave, Houdini, Maya, modo, MotionBuilder, Softimage, the visual effects compositor Nuke, 2D imaging programs like GIMP, Inkscape, Scribus and Paint Shop Pro, and musical notation programs like scorewriter and capella. GNU Debugger uses Python as a pretty printer to show complex structures such as C++ containers. Esri promotes Python as the best choice for writing scripts in ArcGIS. It has also been used in several video games, and has been adopted as first of the three available programming languages in Google App Engine, the other two being Java and Go.
- Python is commonly used in artificial intelligence projects with the help of libraries like TensorFlow, Keras, Pytorch and Scikit-learn. As a scripting language with modular architecture, simple syntax and rich text processing tools, Python is often used for natural language processing.
- Many operating systems include Python as a standard component. It ships with most Linux distributions, AmigaOS 4, FreeBSD (as a package), NetBSD, OpenBSD (as a package) and macOS and can be used from the command line (terminal). Many Linux distributions use installers written in Python: Ubuntu uses the Ubiquity installer, while Red Hat Linux and Fedora use the Anaconda installer. Gentoo Linux uses Python in its package management system, Portage.
- Python is used extensively in the information security industry, including in exploit development.

- Most of the Sugar software for the One Laptop per Child XO, now developed at Sugar Labs, is written in Python. The Raspberry Pi single-board computer project has adopted Python as its main user-programming language.
- Due to Python's user-friendly conventions and easy-to-understand language, it is commonly used as an intro language into computing sciences with students. This allows students to easily learn computing theories and concepts and then apply them to other programming languages.
- LibreOffice includes Python, and intends to replace Java with Python. Its Python Scripting Provider is a core feature[169] since Version 4.0 from 7 February 2013.
- Python is a general-purpose, popular programming language, and it is used in almost every technical field. The various areas of Python use are given below.
- Data Science: Data Science is a vast field, and Python is an important language for this field because of its simplicity, ease of use, and availability of powerful data analysis and visualization libraries like NumPy, Pandas, and Matplotlib.
- Desktop Applications: PyQt and Tkinter are useful libraries that can be used in GUI - Graphical User Interface-based Desktop Applications. There are better languages for this field, but it can be used with other languages for making Applications.
- Console-based Applications: Python is also commonly used to create command-line or console-based applications because of its ease of use and support for advanced features such as input/output redirection and piping.
- Mobile Applications: While Python is not commonly used for creating mobile applications, it can still be combined with frameworks like Kivy or BeeWare to create cross-platform mobile applications.
- Software Development: Python is considered one of the best software-making languages. Python is easily compatible with both from Small Scale to Large Scale software.
- Artificial Intelligence: AI is an emerging Technology, and Python is a perfect language for artificial intelligence and machine learning because of the availability of powerful libraries such as TensorFlow, Keras, and PyTorch.

- Web Applications: Python is commonly used in web development on the backend with frameworks like Django and Flask and on the front end with tools like JavaScript HTML and CSS.
- Enterprise Applications: Python can be used to develop large-scale enterprise applications with features such as distributed computing, networking, and parallel processing.
- 3D CAD Applications: Python can be used for 3D computer-aided design (CAD) applications through libraries such as Blender.
- Machine Learning: Python is widely used for machine learning due to its simplicity, ease of use, and availability of powerful machine learning libraries.
- Computer Vision or Image Processing Applications: Python can be used for computer vision and image processing applications through powerful libraries such as OpenCV and Scikit-image.
- Speech Recognition: Python can be used for speech recognition applications through libraries such as SpeechRecognition and PyAudio.
- Scientific computing: Libraries like NumPy, SciPy, and Pandas provide advanced numerical computing capabilities for tasks like data analysis, machine learning, and more.
- Education: Python's easy-to-learn syntax and availability of many resources make it an ideal language for teaching programming to beginners.
- Testing: Python is used for writing automated tests, providing frameworks like unit tests and pytest that help write test cases and generate reports.
- Gaming: Python has libraries like Pygame, which provide a platform for developing games using Python.
- IoT: Python is used in IoT for developing scripts and applications for devices like Raspberry Pi, Arduino, and others.
- Networking: Python is used in networking for developing scripts and applications for network automation, monitoring, and management.
- DevOps: Python is widely used in DevOps for automation and scripting of infrastructure management, configuration management, and deployment processes.
- Finance: Python has libraries like Pandas, Scikit-learn, and Statsmodels for financial modeling and analysis.

- Audio and Music: Python has libraries like Pyaudio, which is used for audio processing, synthesis, and analysis, and Music21, which is used for music analysis and generation.
- Writing scripts: Python is used for writing utility scripts to automate tasks like file operations, web scraping, and data processing.Python has wide range of libraries and frameworks widely used in various fields such as machine learning, artificial intelligence, web applications, etc. We define some popular frameworks and libraries of Python as follows.
- Web development (Server-side) - Django Flask, Pyramid, CherryPy
- GUIs based applications - Tkinter, PyGTK, PyQt, PyJs, etc.
- Machine Learning - TensorFlow, PyTorch, Scikit-learn, Matplotlib, Scipy, etc.
- Mathematics - NumPy, Pandas, etc.
- BeautifulSoup: a library for web scraping and parsing HTML and XML
- Requests: a library for making HTTP requests
- SQLAlchemy: a library for working with SQL databases
- Kivy: a framework for building multi-touch applications
- Pygame: a library for game development
- Pytest: a testing framework for Python Django
- REST framework: a toolkit for building RESTful APIs
- FastAPI: a modern, fast web framework for building APIs
- Streamlit: a library for building interactive web apps for machine learning and data science
- NLTK: a library for natural language processing

Python Functions

- Lambda Function - A lambda function is a small, anonymous function that can take any number of arguments but can only have one expression. Lambda functions are often used in functional programming to create functions "on the fly" without defining a named function.
- Recursive Function - A recursive function is a function that calls itself to solve a problem. Recursive functions are often used in functional programming to perform complex computations or to traverse complex data structures.

- Map Function - The map() function applies a given function to each item of an iterable and returns a new iterable with the results. The input iterable can be a list, tuple, or other.
- Filter Function - The filter() function returns an iterator from an iterable for which the function passed as the first argument returns True. It filters out the items from an iterable that do not meet the given condition.
- Reduce Function - The reduce() function applies a function of two arguments cumulatively to the items of an iterable from left to right to reduce it to a single value.
- functools Module - The functools module in Python provides higher-order functions that operate on other functions, such as partial() and reduce().
- Currying Function - A currying function is a function that takes multiple arguments and returns a sequence of functions that each take a single argument.
- Memoization Function - Memoization is a technique used in functional programming to cache the results of expensive function calls and return the cached Result when the same inputs occur again.
- Threading Function - Threading is a technique used in functional programming to run multiple tasks simultaneously to make the code more efficient and faster.

Python Modules

- Math: Gives users access to mathematical constants and pi and trigonometric functions.
- Datetime: Provides classes for a simpler way of manipulating dates, times, and periods.
- OS: Enables interaction with the base operating system, including administration of processes and file system activities.
- Random: The random function offers tools for generating random integers and picking random items from a list.
- JSON: JSON is a data structure that can be encoded and decoded and is frequently used in online APIs and data exchange. This module allows dealing with JSON.
- Re: Supports regular expressions, a potent text-search and text-manipulation tool.

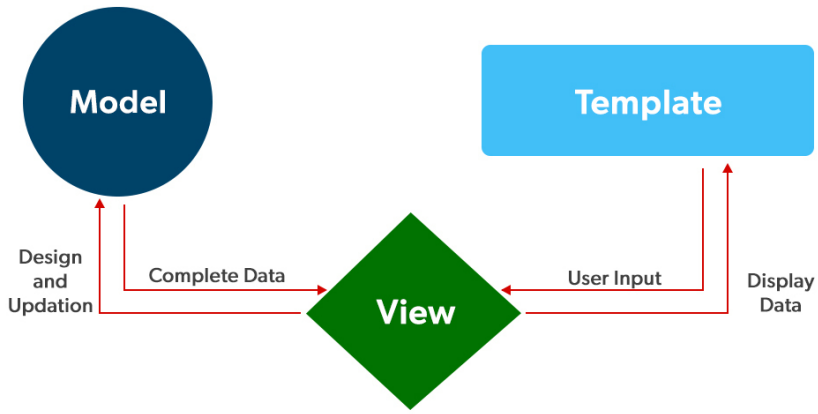
- Collections: Provides alternative data structures such as sorted dictionaries, default dictionaries, and named tuples.
- NumPy: NumPy is a core toolkit for scientific computing that supports numerical operations on arrays and matrices.
- Pandas: It provides high-level data structures and operations for dealing with time series and other structured data types.
- Requests: Offers a simple user interface for web APIs and performs HTTP requests.

Python File I/O

- Files are used to store data in a computer disk. In this tutorial, we explain the built-in file object of Python. We can open a file using Python script and perform various operations such as writing, reading, and appending. There are various ways of opening a file. We are explained with the relevant example. We will also learn to perform read/write operations on binary files.
- Python's file input/output (I/O) system offers programs to communicate with files stored on a disc. Python's built-in methods for the file object let us carry out actions like reading, writing, and adding data to files.
- The open() method in Python makes a file object when working with files. The name of the file to be opened and the mode in which the file is to be opened are the two parameters required by this function. The mode can be used according to work that needs to be done with the file, such as "r" for reading, "w" for writing, or "a" for attaching.
- After successfully creating an object, different methods can be used according to our work.
- If we want to write in the file, we can use the write() functions, and if you want to read and write both, then we can use the append() function and, in cases where we only want to read the content of the file we can use read() function.
- Binary files containing data in a binary rather than a text format may also be worked with using Python. Binary files are written in a manner that humans cannot directly understand. The rb and wb modes can read and write binary data in binary files.

Django Framework

- Django is a web application framework written in Python programming language. It is based on MVT (Model View Template) design pattern. The Django is very demanding due to its rapid development feature. It takes less time to build application after collecting client requirement. This framework uses a famous tag line:The web framework for perfectionists with deadlines.
- By using Django, we can build web applications in very less time.
- Django is designed in such a manner that it handles much of configure things automatically, so we can focus on application development only.
- Python Django is a web framework that allows to quickly create efficient web pages. Django is also called batteries included framework because it provides built-in features such as Django Admin Interface, default database – SQLite3, etc. When you're building a website, you always need a similar set of components: a way to handle user authentication (signing up, signing in, signing out), a management panel for your website, forms, a way to upload files, etc. Django gives you ready-made components to use.Django Architecture
- Django is based on MVT (Model-View-Template) architecture which has the following three parts –
- **Model:** The model is going to act as the interface of your data. It is responsible for maintaining data. It is the logical data structure behind the entire application and is represented by a database (generally relational databases such as MySql, Postgres).
- **View:** The View is the user interface that you see in your browser when you render a website. It is represented by HTML/CSS/Javascript and Jinja files.
- **Template:** A template consists of static parts of the desired HTML output as well as some special syntax describing how dynamic content will be inserted. To check more, visit – Django Templates.



Source Code

```
from django.shortcuts import render

from django.shortcuts import render, redirect

from django.contrib import messages

from .models import UserRegistration

from .models import Dataset

from django.contrib import messages

from .models import Dataset, ModelTrainingResult

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.svm import SVC

import joblib

import os

from .models import ModelTrainingResult, PredictionResults

import pandas as pd

import numpy as np

# Create your views here.

def index(request):

    return render(request, 'index.html')

# Register

def register(request):

    if request.method == "POST":

        username = request.POST['username']

        email = request.POST['email']

        mobile = request.POST['mobile']

        address = request.POST['address']

        password = request.POST['password']

                if UserRegistration.objects.filter(username=username).exists():

            messages.error(request, "Username already exists!")

            return redirect('register')

        if UserRegistration.objects.filter(email=email).exists():

            messages.error(request, "Email already exists!")

            return redirect('register')
```

```
        #hashed_password = make_password(password)  # Hash the password before
saving
        user = UserRegistration.objects.create(
            username=username,
            email=email,
            mobile=mobile,
            address=address,
            password=password
        )
        messages.success(request, "Registration successful! Please login.")
        return redirect('login')
    return render(request, 'register.html')
def user_login(request):
    if request.method == "POST":
        username = request.POST['username'].strip()
        password = request.POST['password'].strip()
        try:
            # Directly check username and password in DB
            user = UserRegistration.objects.get(username=username, password=password)
            # Save session
            request.session['user_id'] = user.id
            request.session['username'] = user.username
            return redirect('home')
```

```
        except UserRegistration.DoesNotExist:
            messages.error(request, "Invalid username or password!")
            return redirect('login')
    return render(request, 'login.html')
# User Home
def userhome(request):
    if 'user_id' not in request.session:
        return redirect('userApp:login')
    username = request.session.get('username')
    return render(request, 'Userhome.html', {'username': username})
# Logout
def user_logout(request):
    request.session.flush()
    return redirect('login')
def load_dataset(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        description = request.POST.get('description')
        file = request.FILES.get('file')
        if name and file:
            Dataset.objects.create(name=name, description=description, file=file)
            messages.success(request, 'Dataset uploaded successfully!')
            return redirect('load_dataset')
```

```
        else:
            messages.error(request, 'Please provide a name and select a file.')
    datasets = Dataset.objects.all().order_by('-uploaded_at')
    return render(request, 'loaddataset.html', {'datasets': datasets})
from django.shortcuts import render, redirect
from django.contrib import messages
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
def train_models(request):
    datasets = Dataset.objects.all()
    if request.method == "POST":
        dataset_id = request.POST.get('dataset')
        try:
            dataset_obj = Dataset.objects.get(id=dataset_id)
        except Dataset.DoesNotExist:
            messages.error(request, "Selected dataset does not exist.")
            return redirect('train_models')
```

```
        # Delete all previous training results
ModelTrainingResult.objects.all().delete()
        # Load CSV dataset
df = pd.read_csv(dataset_obj.file.path)
# Check if 'Flood' column exists
if 'Flood' not in df.columns:
    messages.error(request, "Dataset must contain a 'Flood' column.")
    return redirect('train_models')
# Encode categorical columns
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
X = df.drop('Flood', axis=1)
y = df['Flood']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Define models
models = {
    "SVM": SVC(),
    "Logistic Regression": LogisticRegression(max_iter=500),
```

```
        "Decision Tree": DecisionTreeClassifier(),
        "Random Forest": RandomForestClassifier()
    }
    # Clear previous results for this dataset
    ModelTrainingResult.objects.filter(dataset=dataset_obj).delete()
    # Train models and save results
    for model_name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred, average='weighted',
zero_division=0)
        recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
        f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
        ModelTrainingResult.objects.create(
            dataset=dataset_obj,
            model_name=model_name,
            accuracy=accuracy,
            precision=precision,
            recall=recall,
            f1_score=f1
        )
    messages.success(request, "Models trained and results saved successfully!")
```

```
        return redirect('train_models')

    # Show existing training results

    training_results = ModelTrainingResult.objects.select_related('dataset').order_by('-
trained_at')

    context = {'datasets': datasets, 'training_results': training_results}

    return render(request, 'trainmodel.html', context)

def model_results_view(request):

    # Fetch all training results

    results = ModelTrainingResult.objects.select_related('dataset').order_by('-
trained_at')

    # Prepare data for Chart.js

    labels = [res.model_name for res in results]

    accuracy_data = [round(res.accuracy*100, 2) for res in results]

    precision_data = [round(res.precision*100, 2) for res in results]

    recall_data = [round(res.recall*100, 2) for res in results]

    f1_data = [round(res.f1_score*100, 2) for res in results]

    context = {

        'results': results,

        'labels': labels,

        'accuracy_data': accuracy_data,

        'precision_data': precision_data,

        'recall_data': recall_data,

        'f1_data': f1_data
```

```
    }
    return render(request, 'trainmodel_results.html', context)
def predict_view(request):
    predicted_result = None  # For template display
    if request.method == "POST":
        try:
            # Get input values from form
            rainfall_mm = float(request.POST.get('rainfall_mm'))
            river_level_m = float(request.POST.get('river_level_m'))
            soil_type = request.POST.get('soil_type')
            slope = float(request.POST.get('slope'))
            population_density = float(request.POST.get('population_density'))
            distance_to_river_km = float(request.POST.get('distance_to_river_km'))
            land_use = request.POST.get('land_use')
            # Prepare input DataFrame exactly as trained
            input_df = pd.DataFrame([{
                'Rainfall_mm': rainfall_mm,
                'River_Level_m': river_level_m,
                'Soil_Type': soil_type,
                'Slope': slope,
                'Population_Density': population_density,
                'Distance_to_River_km': distance_to_river_km,
                'Land_Use': land_use
```

```
        }])
        # Encode categorical features exactly as in training
        for col in ['Soil_Type', 'Land_Use']:
            le = LabelEncoder()
            le.fit(['Sandy', 'Clay', 'Loamy'] if col=='Soil_Type' else ['Urban',
'Agriculture', 'Forest'])
            input_df[col] = le.transform(input_df[col])
        # Load trained Random Forest model
        model_path = os.path.join('media', 'trained_models',
'random_forest_model.pkl')
        if not os.path.exists(model_path):
            messages.error(request, "Random Forest model not found. Train the model
first.")
            return redirect('train_models')
        rf_model = joblib.load(model_path)
        # Predict
        prediction = rf_model.predict(input_df)[0]
        predicted_result = "Flood Zone" if prediction == 1 else "Non-Flood Zone"
        # Save prediction to database
        PredictionResults.objects.create(
            user=request.user if request.user.is_authenticated else None,
            rainfall_mm=rainfall_mm,
            river_level_m=river_level_m,
            soil_type=soil_type,
```

```
            slope=slope,
            population_density=population_density,
            distance_to_river_km=distance_to_river_km,
            land_use=land_use,
            predicted_flood=prediction
        )
        messages.success(request, f"Prediction completed: {predicted_result}")
    except Exception as e:
        print("Prediction Error:", str(e))
        messages.error(request, f"Error during prediction: {str(e)}")
    return render(request, 'predict.html', {'predicted_result': predicted_result})
def predicted_history_view(request):
    # Use 'predicted_at' field instead of 'created_at'
    predicted_results = PredictionResults.objects.all().order_by('-predicted_at')
    context = {'predicted_results': predicted_results}
    return render(request, 'predicted_history.html', context)
def predicted_data_ratio_view(request):
    flood_count = PredictionResults.objects.filter(predicted_flood=1).count()
    non_flood_count = PredictionResults.objects.filter(predicted_flood=0).count()
    total_count = flood_count + non_flood_count

    flood_ratio = round((flood_count / total_count) * 100, 2) if total_count > 0 else 0
```

```
    non_flood_ratio = round((non_flood_count / total_count) * 100, 2) if total_count >
0 else 0

    context = {

        'flood_count': flood_count,

        'non_flood_count': non_flood_count,

        'total_count': total_count,

        'flood_ratio': flood_ratio,

        'non_flood_ratio': non_flood_ratio

    }

    return render(request, 'predictedDataRatio.html', context)
```

8. RESULT AND ANALYSIS

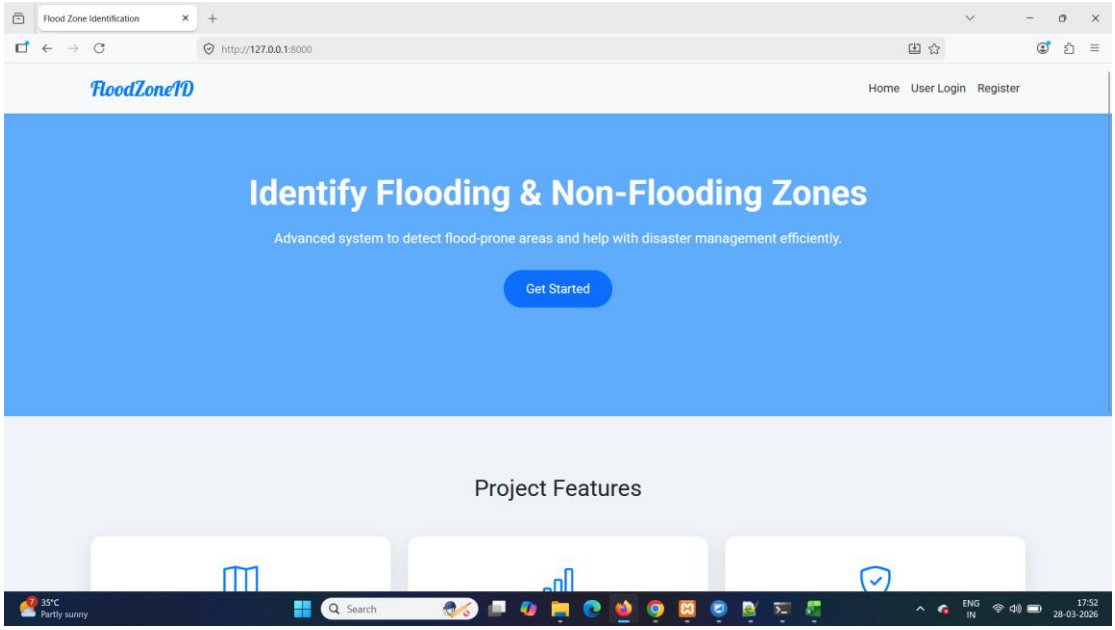


Fig 8.1: Home Page

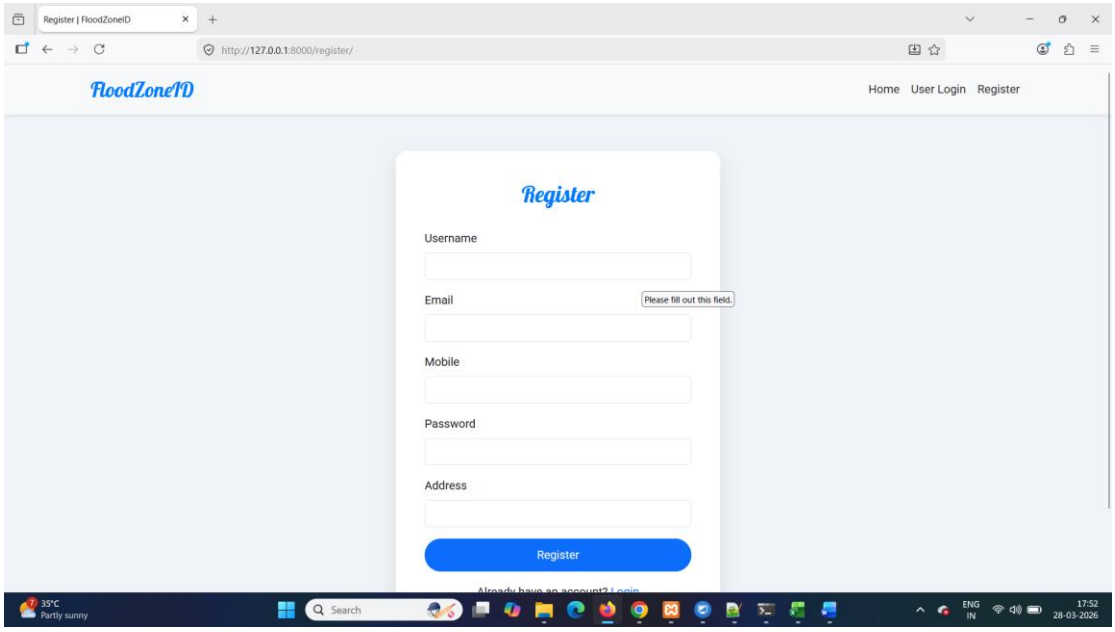


Fig 8.2: Registration

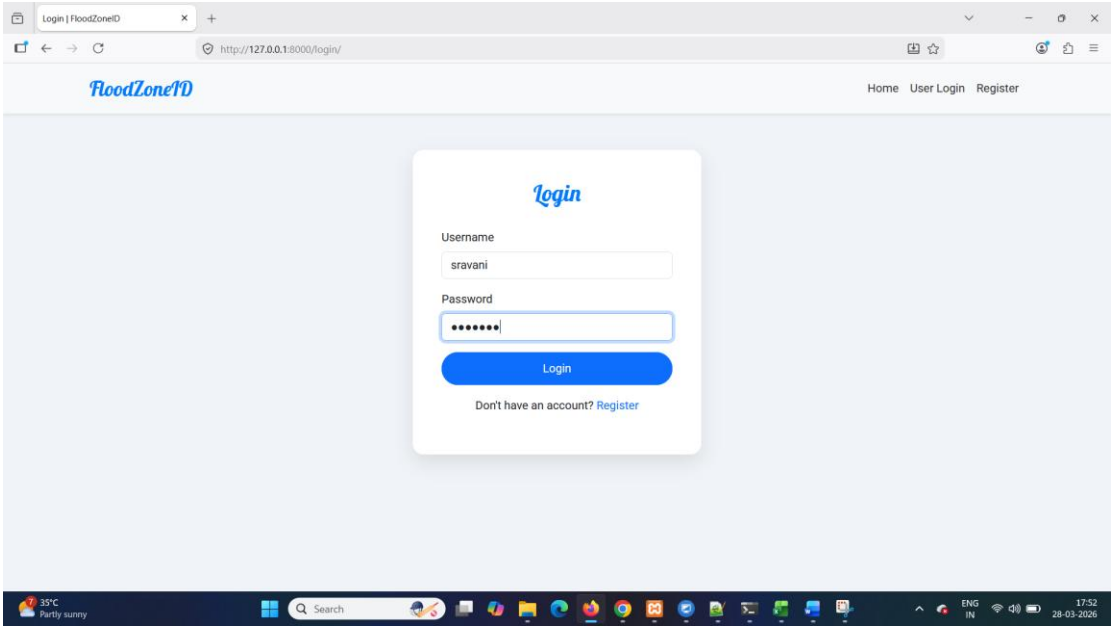


Fig 8.3: Login

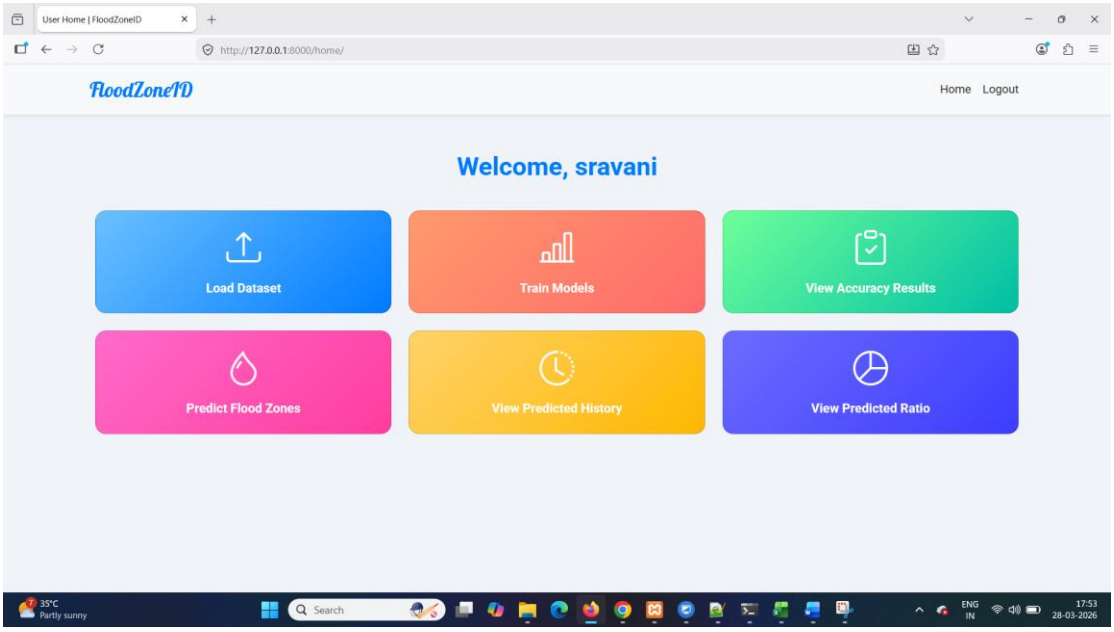


Fig 8.4: Dashboard

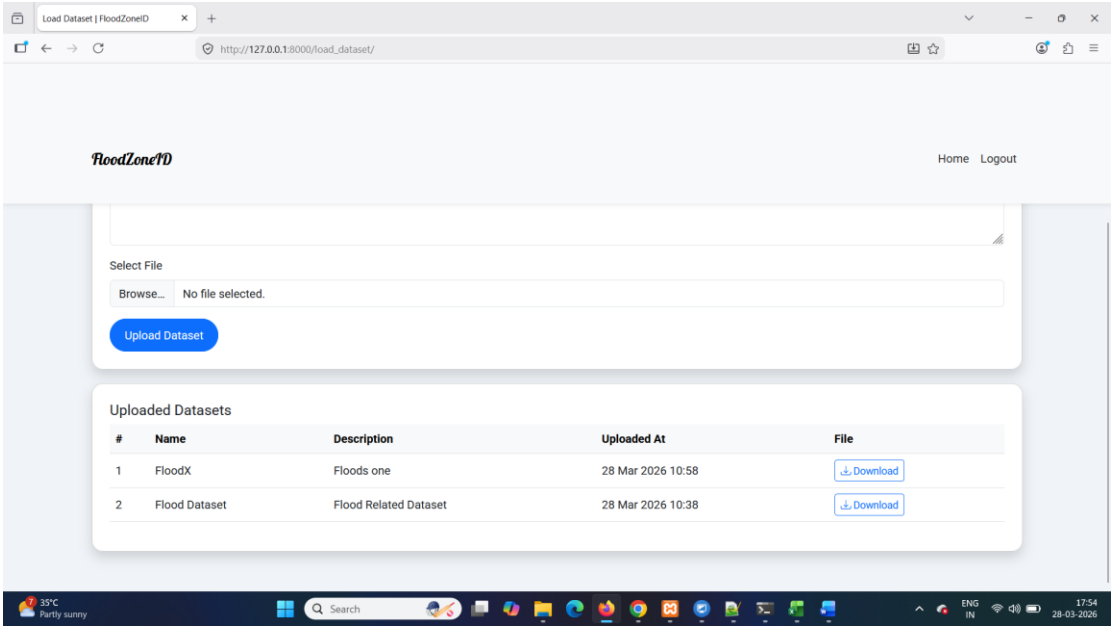


Fig 8.5: Upload Dataset

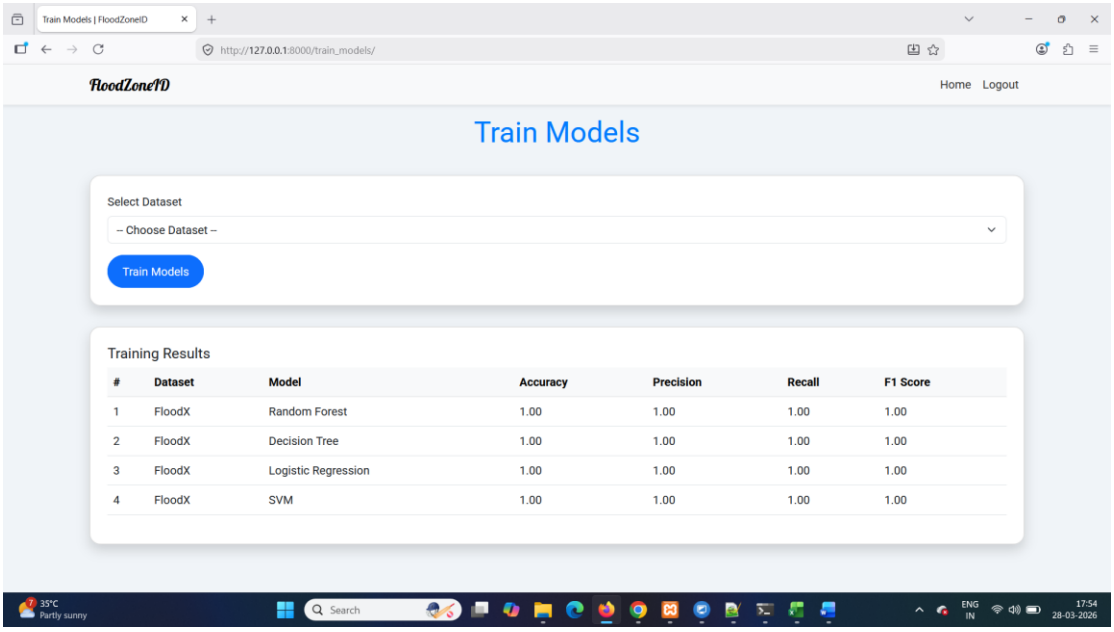


Fig 8.6: Train Models

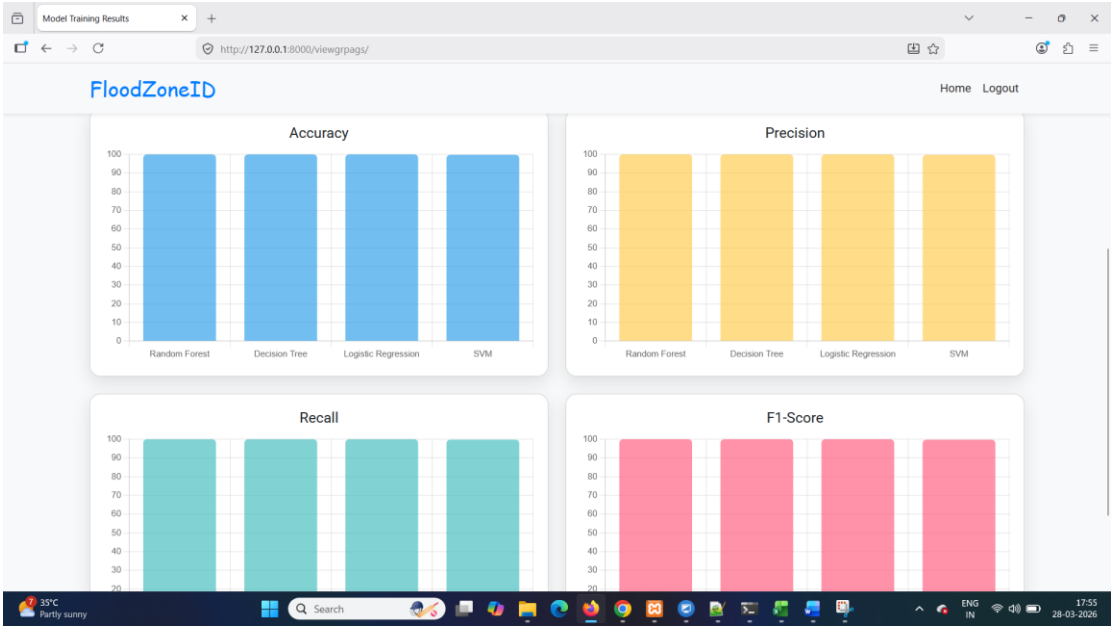


Fig 8.7: Model Accuracy Results

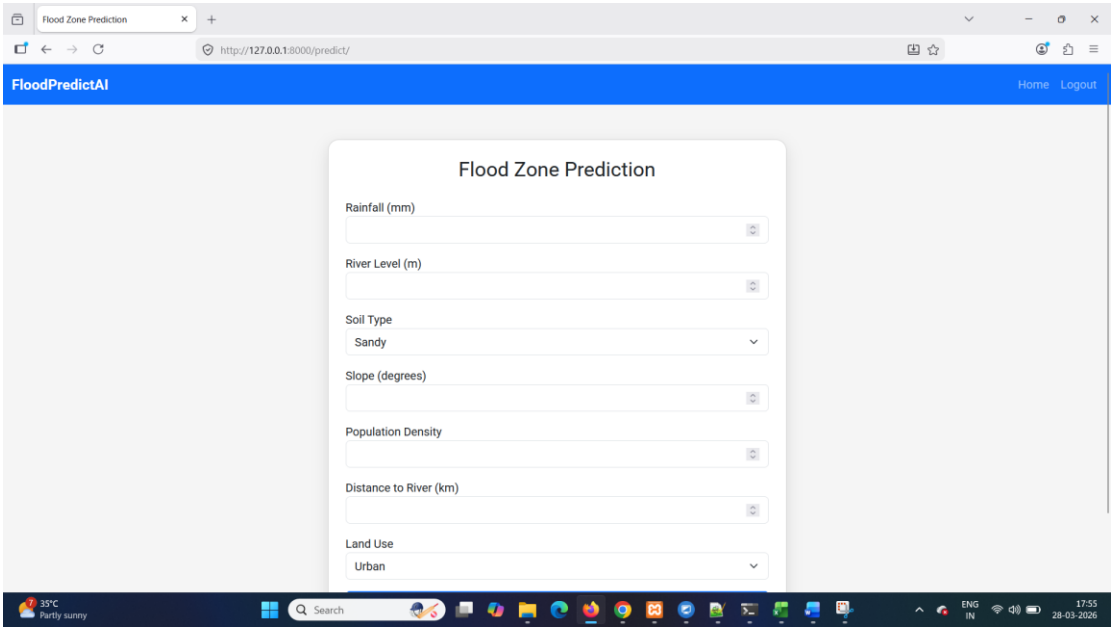


Fig 8.8: Prediction

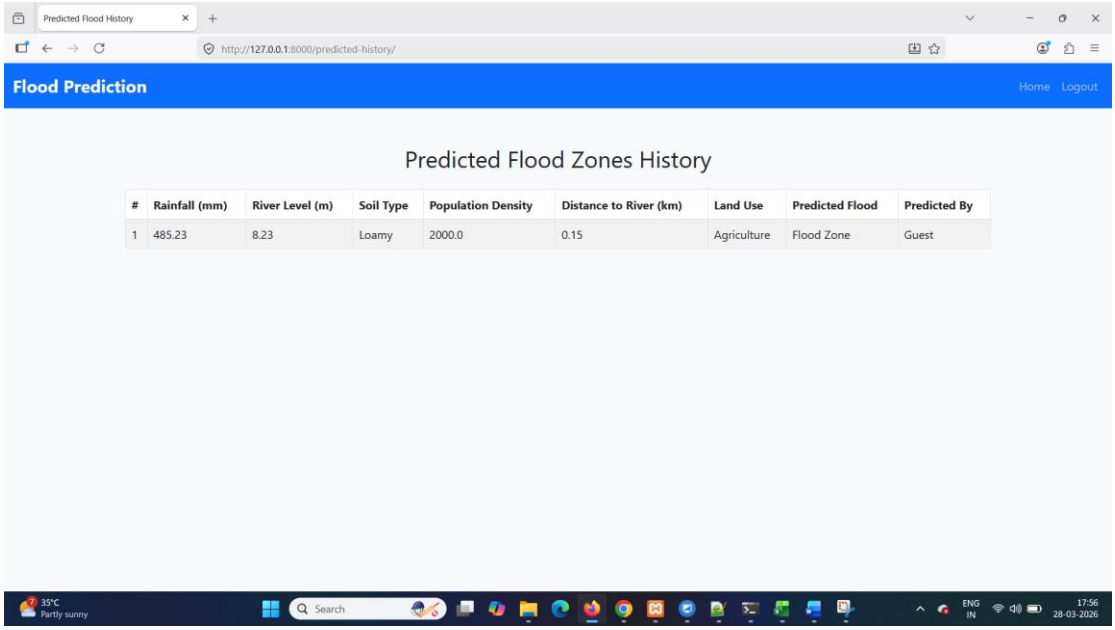


Fig 8.9: Prediction History

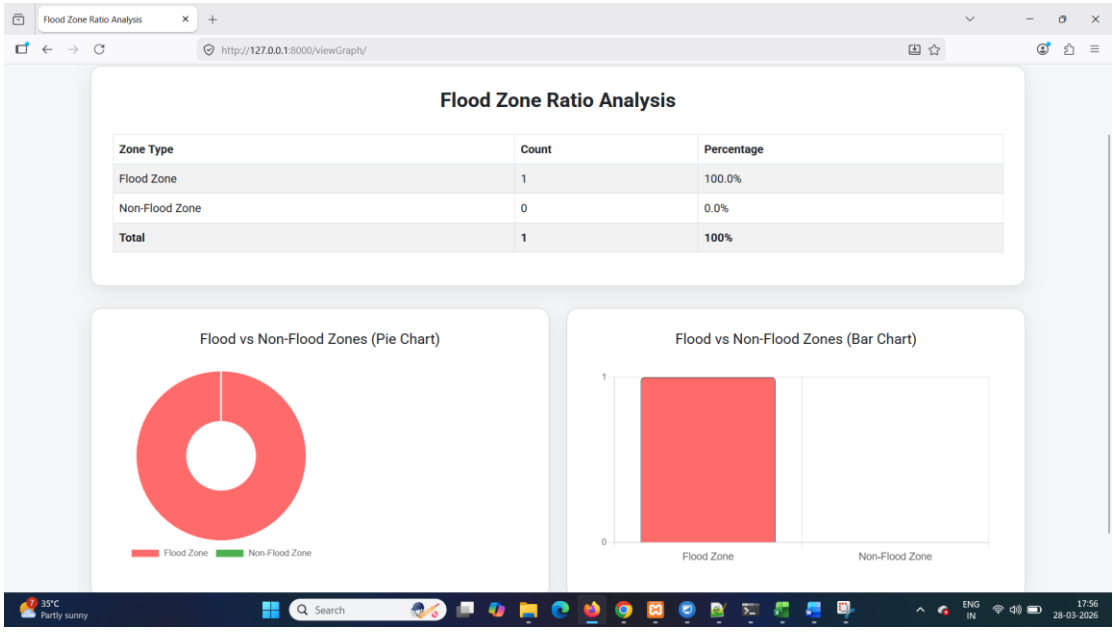


Fig 8.10: Prediction Results

9. CONCLUSION

The purpose of this work is to find out the flooding and non-flooding zones in Bangladesh's four crucial regions such as Sylhet, Barisal, Khulna, and Rangpur. Because intense monsoon seasons cause massive floods that impact millions of people, so we aimed to indicate the flooding areas as possible and mitigate the devastating effects. By Topological Data Analysis (TDA), clustering, Wavelet, and silhouette score we locate the vulnerable places in four places. The combination of this methodology is a new approach in this platform. After using Wavelet in the data set, we were able to denoise the data set which helped to improve our model performance and accuracy. We got the best cluster group k = 4 (between k = 2 and k = 8) and the silhouette score was also high compared to the previous study. The clustering was able to detect unusual feature patterns that were relevant to the detection and classification of flood hazards. This study helps a lot in our country to assuage the devastating effects and is capable of knowing the upcoming danger. The hydrological areas of Bangladesh were successfully classified as flood risk and vulnerability using the vital information obtained from the findings. The unusual pattern discovered in this study was observed to have a positive relationship with the elevation of the zone(s) it occurred. Every flood section produces a feature that is useful in predicting flood and no flood zones. The four parameters involved increased the data points and contributed to the high performance (96%) of the model. All regions exhibited high silhouette values, indicating potential flood-prone areas, with no distinct non-flooding zones identified.

10. BIBLIOGRAPHY

[1] D. L. Mallick, A. Rahman, M. Alam, A. S. M. Juel, A. N. Ahmad, and S. S. Alam,‘‘Casestudy3:BangladeshFloodsinBangladesh:Ashiftfrom disaster management towards disaster preparedness,’’ IDS Bull., vol. 36, no. 4, pp. 53–70, Oct. 2005, doi: 10.1111/j.1759-5436.2005.tb00234.x.

[2] H. Brammer, ‘‘After the Bangladesh flood action plan: Looking to the future,’’ Environ. Hazards, vol. 9, no. 1, pp. 118–130, Mar. 2010, doi: 10.3763/ehaz.2010.si01.

[3] T. H. Dewan, ‘‘Societal impacts and vulnerability to floods in Bangladesh and Nepal,’’ Weather Climate Extremes, vol. 7, pp. 36–42, Mar. 2015, doi: 10.1016/j.wace.2014.11.001.

[4] F. O. Ohanuba, M. T. Ismail, and M. K. M. Ali, ‘‘Topological data anal ysis via unsupervised machine learning for recognizing atmospheric river patterns on flood detection,’’ Sci. Afr., vol. 13, Sep. 2021, Art. no. e00968, doi: 10.1016/j.sciaf.2021.e00968.

[5] M. Nhangumbe, A. Nascetti, S. Georganos, and Y. Ban, ‘‘Supervised and unsupervised machine learning approaches using sentinel data for f lood mapping and damage assessment in Mozambique,’’ Remote Sens. Applications: Soc. Environ., vol. 32, Nov. 2023, Art. no. 101015, doi: 10.1016/j.rsase.2023.101015.

[6] G. Muszynski, K. Kashinath, V. Kurlin, M. Wehner, and Prabhat, ‘‘Topo logical data analysis and machine learning for recognizing atmospheric river patterns in large climate datasets,’’ Geosci. Model Develop., vol. 12, no. 2, pp. 613–628, Feb. 2019, doi: 10.5194/gmd-12-613-2019.

[7] K.K.Zander, D.Nguyen,M.Mirbabaie, and S. T. Garnett, ‘‘Aware but not prepared: Understanding situational awareness during the century flood in Germany in 2021,’’ Int. J. Disaster Risk Reduction, vol. 96, Oct. 2023, Art. no. 103936, doi: 10.1016/j.ijdrr.2023.103936.

[8] X. Jiang, S. Liang, X. He, A. D. Ziegler, P. Lin, M. Pan, D. Wang, J. Zou, D. Hao, G. Mao, Y. Zeng, J. Yin, L. Feng, C. Miao, E. F. Wood, and Z. Zeng, ‘‘Rapid and

large-scale mapping of flood inundation via integrating spaceborne synthetic aperture radar imagery with unsupervised deep learning,'' ISPRS J. Photogramm. Remote Sens., vol. 178, pp. 36–50, Aug. 2021, doi: 10.1016/j.isprsjprs.2021.05.019.

[9] A. H. Tanim, C. B. McRae, H. Tavakol-Davani, and E. Goharian, ''Flood detection in urban areas using satellite imagery and machine learning,'' Water, vol. 14, no. 7, p. 1140, Apr. 2022, doi: 10.3390/w14071140.

[10] G.Parvin,A.Shimi,R.Shaw,andC.Biswas,''Floodinachangingclimate: The impact on livelihood and how the rural poor cope in Bangladesh,'' Climate, vol. 4, no. 4, p. 60, Dec. 2016, doi: 10.3390/cli4040060.

[11] R. Islam and P. Chowdhury, ''Local-scale flash flood susceptibility assessment in Northeastern Bangladesh using machine learning algo rithms,'' Environ. Challenges, vol. 14, Jan. 2024, Art. no. 100833, doi: 10.1016/j.envc.2023.100833.

[12] K. Uddin and M. A. Matin, ''Potential flood hazard zonation and flood shelter suitability mapping for disaster risk mitigation in Bangladesh using geospatial technology,'' Prog. Disaster Sci., vol. 11, Oct. 2021, Art. no. 100185, doi: 10.1016/j.pdisas.2021.100185.

[13] T. Zeng, G. Chen, and G. Reniers, ''Coping with flood natech events: Asafety frameworktomakechemicalclusters moreresilient,'' Chem. Eng. Trans., vol. 90, pp. 805–810, Jan. 2022, doi: 10.3303/CET2290135.

[14] B. Supriyadi, A. P. Windarto, T. Soemartono, and M., ''Classifica tion of natural disaster prone areas in Indonesia using K-means,'' Int. J. Grid Distrib. Comput., vol. 11, no. 8, pp. 87–98, Aug. 2018, doi: 10.14257/ijgdc.2018.11.8.08.

[15] R. R. Kusuma, I. Widianingsih, S. Ningrum, and R. Myrna, ''Five clusters of flood management articles in scopus from 2000 to 2019 using social network analysis,'' Sci. Editing, vol. 8, no. 1, pp. 85–92, Feb. 2021, doi: 10.6087/kcse.234.

[16] I. Irwan, W. Sanusi, A. S. Anwar, and A. Rahman, ''The implementation of spatial model with K-means clustering method to cluster flood affected areas in bone regency,'' ARRUS J. Social Sci. Humanities, vol. 3, no. 2, pp. 186–195, May 2023, doi: 10.35877/soshum1771.

[17] B. Peng, Q. Huang, J. Vongkusolkit, S. Gao, D. B. Wright, Z. N. Fang, and Y. Qiang, ‘‘Urban flood mapping with bitemporal multispectral imagery via a self-supervised learning framework,’’ IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens., vol. 14, pp. 2001–2016, 2021, doi: 10.1109/JSTARS.2020.3047677.

[18] G. I. Drakonakis, G. Tsagkatakis, K. Fotiadou, and P. Tsakalides, ‘‘OmbriaNet–supervised flood mapping via convolutional neural networks using multitemporal Sentinel-1 and Sentinel-2 data fusion,’’ IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens., vol. 15, pp. 2341–2356, 2022, doi: 10.1109/JSTARS.2022.3155559.

[19] E. Arslan and S. Kartal, ‘‘Flood inundation mapping with supervised classifiers: 2021 Gediz plain flood,’’ Turkish J. Remote Sens. GIS, vol. 4, pp. 100–113, Mar. 2023, doi: 10.48123/rsgis.1220879.

[20] J. Vongkusolkit, B. Peng, M. Wu, Q. Huang, and C. G. Andresen, ‘‘Near real-time floodmappingwithweaklysupervisedmachinelearning,’’ RemoteSens.,vol.15,no.13,p. 3263,Jun.2023,doi:10.3390/rs15133263.

[21] F. Chazal and B. Michel, ‘‘An introduction to topological data analysis: Fundamental and practical aspects for data scientists,’’ Frontiers Artif. Intell., vol. 4, Sep. 2021, Art. no. 667963, doi: 10.3389/frai.2021.667963.

[22] A. Karan and A. Kaygun, ‘‘Time series classification via topological data analysis,’’ Expert Syst. Appl., vol. 183, Nov. 2021, Art. no. 115326, doi: 10.1016/j.eswa.2021.115326.

[23] P. D. Bates and A. P. J. De Roo, ‘‘A simple raster-based model for f lood inundation simulation,’’ J. Hydrol., vol. 236, nos. 1–2, pp. 54–77, Sep. 2000, doi: 10.1016/s0022-1694(00)00278-x.

[24] H.Peavy, D. Rowe, andG.Tchobanoglous, Water Resources and Environ mental Engineering, vol. 124. New York, NY, USA: McGraw-Hill, 1985.